Finitary Semantics for Full Ground Local State

ORPHEAS VAN ROOIJ, University of Edinburgh, UK
OHAD KAMMAR, University of Edinburgh, UK
SAM LINDLEY, University of Edinburgh, UK
CRISTINA MATACHE, University of Birmingham, UK

Full ground local state (FGLS) refers to dynamically allocated mutable state that allows storing ground values and references. It is a key ingredient in many imperative algorithms as it enables (cyclic) data structures. In this work, we treat full ground local state as a computational effect, focusing on one particular denotational model: Kammar et al.'s possible worlds monad on sets indexed over sets of locations.

We resolve an outstanding question regarding this FGLS monad: is it finitary? We show that the FGLS monad is not finitary by showing the existence of non-finitary computations in the monad. We then introduce a finitary submonad of Kammar et al.'s monad, give it a concrete description and show that it provides an adequate semantics for FGLS. The submonad we construct paves the way to understanding FGLS in the future via an equational axiomatization suitable for program reasoning.

CCS Concepts: • **Theory of computation** → **Denotational semantics**; *Abstraction.*

Additional Key Words and Phrases: local state, monads, observational equivalence, denotational semantics, computational effects, categorical semantics

1 Introduction

A ubiquitous feature of imperative programs is access to dynamically allocated mutable state: reference-based memory that permits allocation of new memory cells, as well as reading and writing from existing ones. Dynamic allocation of mutable memory is often called *local state* since allocated memory can only be accessed from those parts of the program that were passed the reference. In the case of *full ground* local state (FGLS), cells may store ground values (e.g., booleans, integers) and references, enabling us to represent imperative data structures such as cyclic linked lists. Crucially, full ground local state forbids storing functions which introduce general recursion through Landin's knot [20].

Various models for local state exist in the literature which include among others relational models built via logical relations on nominal-cpos [7, 11], step-indexed syntactic logical relations [3, 9], game semantic models [1, 19, 25], as well as monad semantics based on possible worlds [15, 24, 26].

We are concerned with possible worlds semantics that model heaps and types as sets indexed over sets of locations, together with a suitable state-like monad on the appropriate category. For the ground local state case where only ground values can be stored, heaps are of a contravariant nature: we can select smaller segments of the heap by projecting out some of the cells [24, 26]. In the full ground setting this is no longer the case: projecting out a smaller heap may result in an invalid one since the references stored in the cells may now point to undefined data, i.e., they may be dangling. Kammar et al. [15] address this by equipping heaps with a covariant structure: heaps can be extended with new data modelled via a category of instantiations that carries the data. Their FGLS monad supports:

- *Effect masking:* Computations that do not depend or return references are semantically equivalent to pure values.
- *Adequacy:* Denotationally equal terms are observationally indistinguishable.

$$[\![M]\!] = [\![N]\!] \implies M \cong_{\text{obs}} N$$

Authors' Contact Information: Orpheas van Rooij, University of Edinburgh, UK, orpheas.vanrooij@ed.ac.uk; Ohad Kammar, University of Edinburgh, UK, ohad.kammar@ed.ac.uk; Sam Lindley, University of Edinburgh, UK, sam.lindley@ed.ac.uk; Cristina Matache, University of Birmingham, UK, c.matache@bham.ac.uk.

where M, N are λ-terms that manipulate full ground references.

- *Program Equivalences:* Validates desired equations such as

$$\textbf{letref}\, x := V \,\textbf{in}\, {}^{x}M \;\equiv\; M \qquad\qquad x := V; z := {}^{x}W; M \;\equiv\; z := W; x := {}^{z}V; M \tag{1}$$

where $\textbf{letref}\, x := N \,\textbf{in}\, M$ allocates a new reference x in M with value V, $X := V$ updates the contents of reference x with V and ${}^{x}M$ states that x does not occur free in M.

These properties make it a useful model for FGLS and justify using it as basis for more sophisticated stores. This paper extends the metatheory of this monad by resolving an outstanding question.

Finitary presentations of a monad. In seminal work, Moggi used monads to model computational effects such as mutable state, input/output and non-determinism. The theory of algebraic effects e.g. [4, 26] refines Moggi's programme by using algebraic theories in the sense of universal algebra to characterise computational effects equationally. In this setting, the effectful behaviour of programs is described by a set of effect operations and equations between these operations. For example, non-determinism can be described by an operation $\mathrm{or}(x, y)$ that is commutative, associative and idempotent.

Moggi's monads can be recovered via the well-known correspondence between monads and algebraic theories [13, 21, 29]: finitary algebraic theories correspond to finitary monads on the category of sets. An algebraic theory is finitary when it admits a presentation whose operations are of finite arity. There is no requirement that the set of operations or equations are finite.

However, when describing a computational effect, one often seeks a presentation of the algebraic theory with a small number of intuitive operations and equations, as in the non-determinism example above. The role of the presentation is to serve as a basis for programming with the effect, via the operations, and for reasoning about effectul programs via the equations.

The goal of our line of research is to find such an algebraic presentation for the full ground local state monad of Kammar et al., for which none has been discovered yet. Roughly speaking, the FGLS monad is defined on indexed sets, rather than sets, so we would need to use a more sophisticated notion of algebraic theory, rather than the classic one, to maintain the monad-theory correspondence.

Such extensions of algebraic theories and their correspondence to monads have been developed, for example: monads with arities [8], enriched Lawvere theories [28] and parameterized algebraic theories [32]. Out of these extensions, only parametrized algebraic theories admit a syntax for specifying presentations of theories, which we are particularly interested in from a programming perspective. Moreover, both enriched Lawvere theories and parameterized algebraic theories require the monad in question to be at least finitary.

Until now, the question whether the full ground local state monad of Kammar et al. is finitary remained open. In this paper, we provide a negative answer. We then characterize a finitary submonad of the FGLS monad and give it a concrete description. Therefore, the submonad we introduce opens the way to characterizing FGLS as a parameterized algebraic theory, or as an enriched Lawvere theory, in future work. The main contributions of the paper are as follows:

(i) *Non-finitarity.* We show that the FGLS monad is non-finitary by showing the existence of certain non-finitary computations (§ 2.3). Such computations cannot be represented as terms in the programming language, and so the model has extra information which allows denotations to detect observationally undetectable differences.

(ii) *Worlds, Heaps with variables and Unification.* We develop the machinery needed to describe finitary FGLS computations. We introduce two-kinded worlds, heaps with variables, heap unification and matching and identify the relevant structures (§ 3).

(iii) *Finitary FGLS monad.* Using this machinery we define a finitary FGLS submonad and show that its functor and monad structure lifts from the full monad (§ 4.3). Due to its finitary nature, this monad is more amenable to equational axiomatisations. It further enjoys adequacy of the denotational semantics with respect to the operational semantics, inherited from the non-finitary monad (§ 5).

2 Background and Overview

2.1 Programming Language λ_{ref}

We revisit the programming language λ_{ref}, a higher-order λ-calculus with full ground references. The formalism of λ_{ref} is parametrised by a set of sorts $C \in \mathsf{Sort}$ which we use to define the set of full ground types as

$$\mathsf{FGType} \ni D ::= \mathsf{ref}_C \mid 0 \mid D + D' \mid 1 \mid D \times D'$$

It includes a reference type ref_C for every sort C, the empty type 0 and $A + B$ representing finite sums, and unit type 1 and $A \times B$ representing finite products.

We further require an interpretation function *ctype* that assigns a full ground type D for every sort C. This stratification between sorts and full ground types enables us to represent potentially cyclic data structures where the recursive occurrence is guarded by a reference type.

Example 2.1. Consider $\mathsf{Sort} := \{\mathsf{bool}, \mathsf{boolList}, \mathsf{nat}, \mathsf{nat2}\}$. The following interpretation for bool, boolList represents boxed boolean valued linked lists:

$$\mathit{ctype}\,\mathsf{bool} := 1 + 1 \qquad \mathit{ctype}\,\mathsf{boolList} := 1 + \mathsf{ref}_{\mathsf{bool}} \times \mathsf{ref}_{\mathsf{boolList}}$$

and for nat represents boxed natural numbers and nat2 are boxed pairs of boxed natural numbers:

$$\mathit{ctype}\,\mathsf{nat} := 1 + \mathsf{ref}_{\mathsf{nat}} \qquad \mathit{ctype}\,\mathsf{nat2} := \mathsf{ref}_{\mathsf{nat}} \times \mathsf{ref}_{\mathsf{nat}}$$

Worlds. Pointers to memory cells are called *locations* ranging over a countable set $\mathbb{L}$, and finite sets of locations with an associated sort are called worlds. Formally, a world is a partial function $w : \mathbb{L} \rightharpoonup_{\mathsf{fin}} \mathsf{Sort}$ with finite support. The finite support of w is denoted by $\underline{w}$. We write worlds as $w = l_1 : C_1, \ldots, l_n : C_n$, to really mean its support is $\underline{w} = l_1, \ldots, l_n$ and satisfies $w(l_i) = C_i$. We write $|w|$ for the cardinality of w, which is n in this case, and we order worlds by inclusion: $w \leq w' \iff \forall l : C \in w.\ w'(l) = C$. Lastly, by convention, when enumerating locations $l_1, \ldots, l_n$ we assume that they are all distinct.

Syntax. λ_{ref} is a coarse-grained call-by-value language with the usual λ-calculus constructs for finite sums and products. Figure 1 gives the syntax of λ_{ref} where the shaded constructs relate to references. The types of λ_{ref} are the full ground types closed under the function type constructor, thus full ground types are a subset of all λ_{ref} types. Values include variables and λ-abstractions, location literals l which are pointers to allocated memory cells as above, as well as the unit, binary pairs and injections. Expressions consist of the usual constructs: values, application, pairs over expressions, pair eliminator $\mathbf{split}\, M\, \mathbf{as}\, (x_1, x_2).\, N$, injections over expressions, falsity eliminator $\mathbf{case}\, M\{\}$, and sum eliminator $\mathbf{case}\, M\, \{x_1.\, N_1; x_2.N_2\}$. For references, $M == N$ performs equality testing between two references, $!\, M$ reads a reference, $M := N$ writes value of N to reference M, and $\mathbf{letref}\, x_1 := V_1, \ldots, x_n := V_n\, \mathbf{in}\, M$ allocates n references with the corresponding values and makes them available to M.

We introduce the following sugar: $2 = 1 + 1$ with $\mathbf{true} := \mathit{inj}_1()$, $\mathbf{false} := \mathit{inj}_2()$, $\mathbf{let}\, x = M\, \mathbf{in}\, N := (\lambda x.\, N)M$, and $M; N := \mathbf{let}\, x = M\, \mathbf{in}\, N$ where x does not occur free in N.

Typing judgments $\Gamma \vdash^{\mathsf{v}}_{w} V : A$ for values and $\Gamma \vdash_{w} M : A$ for expressions are parametrised by a world w that represents the allocated locations. This parameter is used in (Loc) to ensure that locations l are present in the prescribed world. The type system is monotone on world extensions

$$
\begin{array}{rll}
\text{Type} \ni A, B ::= & \mathsf{ref}_C \mid 0 \mid A + B \mid 1 \mid A \times B \mid A \to B & \text{(Reference and } \times, +, \to \text{ types)} \\
\text{Val} \ni V, W ::= & x \mid \lambda x.\, M \mid l \mid & (\lambda\text{-calculus, and location literals}) \\
& () \mid (V, W) \mid & \text{(Products)} \\
& \mathbf{inj}_{i \in \{1,2\}}\, V & \text{(Sums)} \\
\text{Expr} \ni M, N ::= & V \mid M\, N \mid & \text{(Values and Application)} \\
& (M, N) \mid \mathbf{split}\, M\, \mathbf{as}\, (x_1, x_2).\, N \mid & \text{(Products)} \\
& \mathbf{inj}_{i \in \{1,2\}}\, M \mid \mathbf{case}\, M \{\} \mid \mathbf{case}\, M \{x_1.\, N_1; x_2.N_2\} \mid & \text{(Sums)} \\
& M == N \mid\ !M \mid\ M := N \mid\ \mathbf{letref}\ x_1 := V_1, \ldots, x_n := V_n\ \mathbf{in}\ M & \text{(References)}
\end{array}
$$

Fig. 1. Syntax of λ_{ref}

$w \leq w'$, thus rule (WorldExt) is admissible. The interpretation function *ctype* is used in (Read), (Write) to obtain the full ground type associated with the sort. Equality testing (Eq) expects two expressions with reference types. Allocation (Alloc) performs simultaneous recursive allocation, that is, makes the references to be allocated available in the instantiation data. This allows the creation of mutually cyclic data structures.

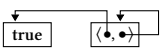
$$
\text{(Loc)}\ \frac{l : C \in w}{\Gamma \vdash^{\mathsf{v}}_w l : \mathsf{ref}_C}
\qquad
\text{(WorldExt)}\ \frac{w \leq w' \qquad \Gamma \vdash_w M : A}{\Gamma \vdash_{w'} M : A}
\qquad
\text{(Read)}\ \frac{\Gamma \vdash_w M : \mathsf{ref}_C}{\Gamma \vdash_w\ !M : \mathit{ctype}\, C}
\qquad
\text{(Write)}\ \frac{\Gamma \vdash_w M : \mathsf{ref}_C \qquad \Gamma \vdash_w N : \mathit{ctype}\, C}{\Gamma \vdash_w M := N : 1}
$$

$$
\text{(Eq)}\ \frac{\Gamma \vdash_w M : \mathsf{ref}_C \qquad \Gamma \vdash_w N : \mathsf{ref}_C}{\Gamma \vdash_w M == N : 2}
\qquad
\text{(Alloc)}\ \frac{(\Gamma, x_1 : \mathsf{ref}_{C_1}, \ldots, x_n : \mathsf{ref}_{C_n} \vdash_w V_i : \mathit{ctype}\, C_i)_{i=1}^{n} \qquad \Gamma, x_1 : \mathsf{ref}_{C_1}, \ldots, x_n : \mathsf{ref}_{C_n} \vdash_w M : A}{\Gamma \vdash_w \mathbf{letref}\ x_1 := V_1, \ldots, x_n := V_n\ \mathbf{in}\ M : A}
$$

Fig. 2. Typing rules of λ_{ref} regarding references

Example 2.2. Creates a cyclic list with one node with the associated heap given on the right

$$\mathbf{letref}\ (x : \mathsf{ref}_{\mathsf{bool}}) := \mathbf{true}, (y : \mathsf{ref}_{\mathsf{boolList}}) := \mathbf{inj}_2\ (x, y)\ \mathbf{in}\ y$$



Operational Semantics. An (untyped) heap h is a partial function $\mathbb{L} \rightharpoonup_{\mathsf{fin}} \mathrm{Val}$, and a configuration $\langle M, \eta \rangle$ consists of an expression M and a heap η. The operational semantics of λ_{ref} is given in big-step, $\langle M, \eta \rangle \Downarrow \langle V, h' \rangle$, which relates configurations on expressions with configurations on values. Figure 3 gives the rules related to references.

(Red-Read) evaluates M to a location l and returns its value provided the resulting heap h' defines l. (Red-Write) evaluates from left to right, reducing M and N to a location l and new value W and updates the heap l with the new value. (Red-Eq) evaluates left to right, reducing M and N to locations l_1, l_2 and returns whether they are equal $l_1 == l_2$. Function $\#(b)$ interprets boolean b from the metalevel to syntax. Rule (Red-Alloc) generates distinct fresh locations $l_1, \ldots, l_n$, and evaluates the body M with the updated heap after substituting the new locations.

$$\text{(Red-Read)}\ \frac{\langle M, h\rangle \Downarrow \langle l, h'\rangle \quad h'(l) = V}{\langle !\, M, h\rangle \Downarrow \langle V, h'\rangle} \qquad \text{(Red-Write)}\ \frac{\langle M, h\rangle \Downarrow \langle l, h'\rangle \quad \langle N, h'\rangle \Downarrow \langle W, h''\rangle}{\langle M := N, h\rangle \Downarrow \langle (), h''[l \mapsto W]\rangle} \qquad \text{(Red-Eq)}\ \frac{\langle M, h\rangle \Downarrow \langle l_1, h'\rangle \quad \langle N, h'\rangle \Downarrow \langle l_2, h''\rangle}{\langle M == N, h\rangle \Downarrow \langle \#(l_1 == l_2), h''\rangle}$$

$$\text{(Red-Alloc)}\ \frac{\theta = [x_i \mapsto l_i]_{i=1}^n \qquad l_1, \ldots, l_n \notin \mathrm{Dom}(h) \qquad h' = h[l_i \mapsto V_i[\theta]]_{i=1}^n \qquad \langle M[\theta], h'\rangle \Downarrow \langle W, h''\rangle}{\langle \textbf{letref}\ x_1 := V_1, \ldots, x_n := V_n\ \textbf{in}\ M, h\rangle \Downarrow \langle W, h''\rangle}$$

Fig. 3. Reduction rules of λ_{ref} for references

Well typed terms in λ_{ref} are sound and total: a closed well-typed term evaluated with a compatible heap reduces to a closed value. We refer the reader to [15] for the full specification of λ_{ref} which we include in the appendix for completeness § A.1.

2.2 Full Ground Local State Monad

We assume familiarity with categories, functors and natural transformations, and explain any further concepts as needed. For a small category $\mathbb{C}$, we denote the category of its set-valued functors $\mathbb{C} \to \mathrm{Set}$ as $[\mathbb{C}, \mathrm{Set}]$, and let $\mathbb{C}(C, -) : \mathbb{C} \to \mathrm{Set}$ be the covariant hom functor, i.e., $\mathbb{C}(C, D)$ is the set of all morphisms with domain C and codomain D in $\mathbb{C}$.

The category $\mathbb{W}$ has the worlds w (§ 2.1) for objects, and morphisms $w \xrightarrow{\sigma} w'$ are world extensions with renamings: σ is an injective function between their supports $\underline{w} \rightarrowtail \underline{w'}$ that preserve the sorts. That is, for every location $l \in \underline{w}$, we have $w(l) = w'(\sigma(l))$. Further, let $w' \ominus \sigma$ be the world of all locations in w' that are not hit by σ: $\underline{w' \ominus \sigma} = \underline{w'} \setminus \mathrm{Img}(\sigma)$, and let $w' \ominus \sigma \xrightarrow{\sigma^c} w'$ be the inclusion.

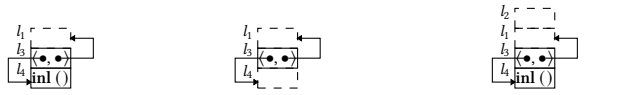
Heaplets and Heaps. Closed full ground values $\cdot \vdash^{\mathrm{v}}_w V : \mathit{ctype}\, C$ are modelled semantically as elements $[\![V]\!] \in [\![\mathit{ctype}\, C]\!](w)$, where $[\![\mathit{ctype}\, C]\!]$ is an indexed set over worlds i.e., a functor in $\mathbf{W} := [\mathbb{W}, \mathrm{Set}]$. The monotonicity of the type system with respect to world extensions (WorldExt) is modelled by the covariant structure of functors $[\![\mathit{ctype}\, C]\!]$. A world extension $w \leq w'$ is a morphism in $\mathbb{W}$, and thus the functorial action on morphisms takes a $[\![V]\!] : [\![\mathit{ctype}\, C]\!](w)$ and extends it to $[\![\mathit{ctype}\, C]\!](w \leq w')[\![V]\!] : [\![\mathit{ctype}\, C]\!](w')$. Reference types are interpreted using the hom-functor $[\![\mathrm{ref}_C]\!] = \mathbb{W}((l : C), -)$, and thus $\sigma \in [\![\mathrm{ref}_C]\!](w)$ is a morphism $(l : C) \xrightarrow{\sigma} w$ in $\mathbb{W}$, i.e., a selection of a single C-sorted location in w.

The *heaplet* functor $\underline{\mathbb{H}} : \mathbb{W}^{\mathrm{op}} \times \mathbb{W} \to \mathrm{Set}$ represents heap segments with w^- defined locations.

$$\underline{\mathbb{H}}(w^-, w^+) = \prod_{l:C \in w^-} [\![\mathit{ctype}\, C]\!](w^+)$$

Values in a heaplet have access to a possibly bigger world w^+, enabling local level reasoning in a spirit similar to separation logic. The heaplet functor is contravariant on world w^- (we can project out a smaller heaplet), and covariant on world w^+ (by the covariant structure of semantic values).

Example 2.3. Consider world extensions $(l_3 : \mathrm{nat2}) = w_1' \leq w_1 = (l_3 : \mathrm{nat2}, l_4 : \mathrm{nat})$ and $(l_1, l_4 : \mathrm{nat}, l_3 : \mathrm{nat2}) = w_2 \leq w_2' = (l_1, l_2, l_4 : \mathrm{nat}, l_3 : \mathrm{nat2})$. Applying them to heaplet η results in:



$\eta : \underline{\mathbb{H}}(w_1, w_2)$ $\quad \underline{\mathbb{H}}(w_1' \leq w_1, w_2)(\eta)$ $\quad \underline{\mathbb{H}}(w_1, w_2 \leq w_2')(\eta)$

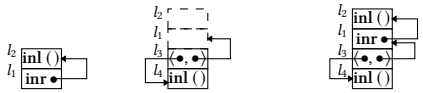
We equip $\mathbb{W}$ with data using heaplets to form the category $\mathbb{WI}$. Objects in $\mathbb{WI}$ are worlds, and morphisms $w \xrightarrow{\varepsilon} w'$ consist of a morphism $u\varepsilon : w \to w'$ in $\mathbb{W}$ and a heaplet $\eta_\varepsilon : \underline{\mathbb{H}}(w' \ominus \sigma, w')$. Thus η_ε defines a value for every location $l : C \in w'$ that is not hit by $u\varepsilon$, with the value having access to the bigger w' world. Composition of morphisms in $\mathbb{WI}$ is further described in § 3.1, and we have a forgetful functor $u : \mathbb{WI} \to \mathbb{W}$.

The heap functor $\mathbb{H} \in \mathbf{WI} := [\mathbb{WI}, \mathrm{Set}]$ represents fully defined heaplets, given on worlds as:

$$\mathbb{H}(w) = \underline{\mathbb{H}}(w, w) \cong \mathbb{WI}(\emptyset, w)$$

The functorial action on morphisms $\mathbb{H}(\varepsilon)(\eta)$ extends a heap η with a heaplet η_ε, giving a covariant structure to full-ground heaps.

Example 2.4. We form morphism $\varepsilon : (l_1, l_2 : \mathrm{nat}) \hookrightarrow w_2'$ in $\mathbb{WI}$ using the heaplet from Example 2.3, $\eta_\varepsilon := \underline{\mathbb{H}}(w_1, w_2 \leq w_2')(\eta)$. Applying ε to heap η_1 given below results in a heap $\mathbb{H}(\varepsilon)(\eta_1)$ where locations l_3, l_4 are not reachable from l_1, l_2.



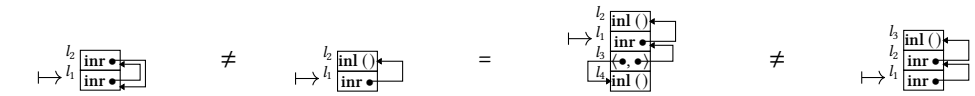
heap η_1 $\quad$ heaplet η_ε $\quad$ heap $\mathbb{H}(\varepsilon)(\eta_1)$

Stateful Values. In the setting of local state, return values are not necessarily pure. They may return an existing or newly allocated reference, and are therefore understood in the context of a world, i.e., a heap layout. Accordingly, *stateful values* are an abstraction over return values that additionally track the allocations that occurred and the resulting state of the heap.

Semantically, stateful values are given via a $\mathbf{W}$-action $(- \odot \mathbb{H})$ on $\mathbf{WI}$ and a monad $(\mathbb{P}, \eta^{\mathbb{P}}, \gg\!=^{\mathbb{P}})$ on $\mathbf{WI}$ shown in Figure 4. Functor $(X \odot \mathbb{H})$ represents indexed sets of pairs over worlds: it lifts $X \in \mathbf{W}$ to $\mathbf{WI}$ using the forgetful functor and takes the cartesian product in $\mathbf{WI}$, $X \odot \mathbb{H} = Xu \times \mathbb{H}$. Monad $\mathbb{P}$ on the other hand models allocation with garbage collection capabilities.

For a return type $X : \mathbb{W} \to \mathrm{Set}$, and world w representing the existing heap layout, a stateful value $q_{w \xrightarrow{\sigma} w'}\langle x', \eta' \rangle \in \mathbb{P}(X \odot \mathbb{H})(w)$ allocates $w' \ominus \sigma$ locations, returns value $x' : X(w')$ and updates the heap to $\eta' : \mathbb{H}(w')$. These stateful values are viewed only up to observable allocations, thus modelling garbage collection.

Example 2.5. All four stateful values are over public world $(l_1, l_2 : \mathrm{nat})$ with $[\![\mathsf{ref}_{\mathsf{nat}}]\!]$ return value:



$q_{l_1, l_2 \leq l_1, l_2 : \mathrm{nat}}\langle l_1, \eta_c \rangle$ $\quad$ $q_{l_1, l_2 \leq l_1, l_2 : \mathrm{nat}}\langle l_1, \eta_1 \rangle$ $\quad$ $q_{l_1, l_2 \leq w_2'}\langle l_1, \mathbb{H}(\varepsilon)(\eta_1) \rangle$ $\quad$ $q_{l_1, l_2 \leq (l_1, l_2, l_3 : \mathrm{nat})}\langle l_1, \eta_2 \rangle$

Only the middle two stateful values are equal because allocations l_3, l_4 is not observable: they are not returned nor are they reachable from l_1, l_2 and are therefore garbage collected. In contrast, the left-most stateful value stores **inr** () at l_2 and in the right-most, allocation l_3 is observable from l_1.

Intuitively, unit $\eta^{\mathbb{P}}$ injects into a context with no allocations and bind $\gg\!=^{\mathbb{P}}$ composes the allocations.



Fig. 4. State-transformer description of the FGLS Monad

The Monad. Action $(- \odot \mathbb{H})$ has a right adjoint $(\mathbb{H} \multimap -)$ by generalities, inducing a state transformer over full ground heaps (Figure 4). We think of $(\mathbb{H} \multimap A)$ as lifting the exponential $A^{\mathbb{H}}$ in **WI** to **W**. More concretely, at world w it is equivalent to:

$$(\mathbb{H} \multimap A)(w) \cong \mathbf{WI}(\mathbb{W}(w, -) \odot \mathbb{H}, A)$$

Specialising this adjunction to stateful values gives us the function-like structure of $\mathcal{T}$:

$$\mathbf{WI}(X \odot \mathbb{H}, \mathbb{P}(Y \odot \mathbb{H})) \underset{\text{uncurry}_{X,Y}}{\overset{\text{curry}_{X,Y}}{\cong}} \mathbf{W}(X, \mathbb{H} \multimap \mathbb{P}(Y \odot \mathbb{H})) \qquad \text{eval} := \text{uncurry}_{\mathcal{T}X,X}(1_{\mathcal{T}X})$$

Therefore, a FGLS computation $\phi \in \mathcal{T}Xw$ is a natural transformation $\mathbb{W}(w, -) \odot \mathbb{H} \to \mathbb{P}(X \odot \mathbb{H})$: defines a stateful value $\phi_{w'}\langle \sigma', \eta' \rangle$ for every $w \xrightarrow{\sigma} w' \in \mathbb{W}$ and heap $\eta' : \mathbb{H}(w')$. Naturality says that ϕ must only examine the reachable parts of the heap starting from world w: different unreachable heap segments should not be observable. Finally, the monad structure of $\mathcal{T}$ is given by the monad $\mathbb{P}$ and curry/uncurry:

$$\eta_X := \text{curry}_{X,X}(\eta^{\mathbb{P}}_{X \odot \mathbb{H}}) \qquad : X \to \mathcal{T}X$$
$$\gg\!=\!(f : X \to \mathcal{T}Y) := \mathbb{H} \multimap (\gg\!=\!^{\mathbb{P}}(\text{uncurry}_{X,Y} f)) : \mathcal{T}X \to \mathcal{T}Y$$

$\eta_X(x)$ is a computation that returns x leaving the heap unchanged (no allocations occur), and $c \gg\!= f$ first evaluates c and calls f with its result value and the modified heap.

We note in passing, that the FGLS monad is equivalently given using an End:

$$\mathcal{T}Xw \cong \int_{w \rightarrowtail w' \in w \downarrow u} \mathbb{H}w' \to \mathbb{P}(X \odot \mathbb{H})(w')$$

where $w \downarrow u$ is the comma category over world $w \in \text{Obj}(\mathbb{W})$ and forgetful functor $u : \mathbb{W}\mathbb{I} \to \mathbb{W}$.

2.3 Non Finitariness of FGLS Monad

As explained in the introduction, one of the long-term goals of our work is finding an equational characterization of FGLS via a suitable notion of algebraic theory. To be able to use the framework of parameterized algebraic theories [32] for this purpose, which provides a syntax for presenting theories, or that of enriched Lawvere theories [28], we would require the FGLS monad to be *finitary*. In this section we discuss what being finitary means and why the FGLS monad does not satisfy this requirement.

We explain what it means for a monad to be finitary through the simpler setting of the global state monad on Set. Let S be a set that represents the state, which may be a single ground value cell (e.g., $S := \mathbb{B}, \mathbb{N}$), or an entire heap (e.g., $S := \mathbb{L} \rightharpoonup_{\text{fin}} \text{Val}$). The global state monad is given as $GS_S(X) = S \to X \times S$, which represents state transforming functions with return values of type X.

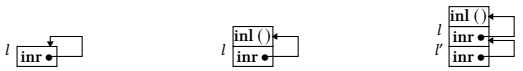
When S is finite, the global state monad is finitary. For instance, $GS_{\mathbb{B}}$ can be equivalently described via an algebraic theory that has operations $get(x, y)$, $put_{\textbf{false}}(x)$, $put_{\textbf{true}}(x)$ where get selects continuation x or y according to the contents of the cell, and put_b updates the value of the store. However for non-finite S, GS_S is not finitary: get would need a countable arity $\mathbb{N}$ to represent state $S := \mathbb{N}$. Put differently, GS_S is finitary if for every $X, f \in GS_S X$, $(\pi_1 \circ f)$ has finite image [18].

To motivate why finitariness fails for the FGLS monad, consider $S := \mathbb{L} \rightharpoonup_{\text{fin}} \text{Val}$ representing the heaps from § 2.1. The global state monad $GS_{\mathbb{L} \rightharpoonup_{\text{fin}} \text{Val}}$ is not finitary for the same reasons as above:

there exists a function $\lambda\eta.\, |\underline{\eta}| \in GS_{\mathbb{L} \rightharpoonup_{\mathsf{fin}} \mathsf{Val}}(\mathbb{N})$ that counts the number of defined locations and has a countable image.

We define an analogous function in the FGLS setting, called *heapSize*. First let the result type be pure $\mathbb{N}$ values given by the constant functor $\Delta_{\mathbb{N}} = w \mapsto \mathbb{N} \in \mathbf{W}$. We define *heapSize* at world $(l : \mathsf{nat})$ representing a single public location of sort nat pointing to the heap.[1]

Recall from (§ 2.2), to give $\mathit{heapSize} \in \mathcal{T}(\Delta_{\mathbb{N}})(l : \mathbb{N})$ is to give a natural transformation $\mathbb{W}(l : \mathsf{nat}, -) \odot \mathbb{H} \to \mathbb{P}(\Delta_{\mathbb{N}} \odot \mathbb{H})$, where naturality says that *heapSize* must only examine the reachable parts of the heap starting from l. Therefore, we define *heapSize* at $(l : \mathsf{nat}) \xrightarrow{\sigma} w$, and heap $\eta : \mathbb{H}(w)$ by traversing η starting from $\sigma(l)$ and returning $\eta^{\mathbb{P}} \langle n, \eta \rangle$ where n is the number of distinct locations it has encountered. For instance *heapSize* returns for the following heaps:



$\mathit{heapSize}(l) = 1$ $\quad$ $\mathit{heapSize}(l) = 2$ $\quad$ $\mathit{heapSize}(l) = 2$

Since η is finite, *heapSize* is well-founded. With this example we will show that the FGLS monad is not finitary as it returns countably-many different values according to the heap it is applied to. We refer the reader to (Lemma A.1) for the formal definition of *heapSize*.

We note that even though finitariness for the FGLS monad may seem restrictive as it places bounds on heap traversals, a programming language that supports complete heap traversals while remaining in the realm of terminating languages is also strange. One might imagine introducing *heapSize* as a built-in operation in the language but that operation either exposes unnecessary lower level details about the runtime or requires an expensive garbage collection operation.

In the next section, and in §4, we develop a finitary version of the monad that can in the future be used as a basis for an equational axiomatization of FGLS. We construct our finitary monad by removing non-finitary computations such as *heapSize* from the original monad.

2.4 Finitary FGLS Monad

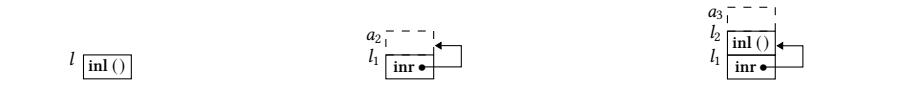
The central idea behind finitary computations is that they return finitely many values. Applying this to the FGLS setting means that it cannot return a different value for every possible heap. We therefore represent a finitary FGLS computation by finitely partitioning the heap space into segments and assigning a stateful value to each segment. We begin by elaborating on the idea of finite partitions of the heap space.

Templates are two-kinded worlds: one kind for concrete locations and another for location variables. While concrete locations represent cells with defined data, location variables are extension points which we can instantiate to extend the heap. We write templates as $\tau = l_1 : C_1, a_2 : C_2, l_3 : C_3$, where metavariable τ ranges over templates, l over concrete locations and a over location variables. We write $\lfloor \tau \rfloor$ for the world with only concrete locations $l_1 : C_1, l_3 : C_3$ and $\lceil \tau \rceil = a_2 : C_2$ for the world with only the variables. We further write $[\tau]$ for the template seen as a world, that is, we forget the distinction given by the two kinds.

Stencils represent collections of heaps with a common set of public locations. That is, for a public world w and template τ, a stencil is a pair $\langle \sigma_s, \eta_s \rangle \in \mathcal{S}_w \tau$, with $\sigma_s : w \to [\tau] \in \mathbb{W}$ giving the mapping of the public locations and heaplet $\eta_s : \underline{\mathbb{H}}(\lfloor \tau \rfloor, [\tau])$ the possibly incomplete segment of the heap. We equip stencils with a covariant structure that extends the heap by instantiating variables $\lceil \tau \rceil$. Thus, a stencil can be thought of as a collection of concrete heaps given by all its extensions.

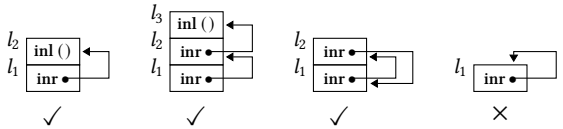
Example 2.6. We define three stencils all over public world $(l : \mathsf{nat})$:

[1] Not all interpretation functions yield a non-finitary FGLS monad, but examples of interest that support cyclic data structures will be non-finitary.



$\tau := (l : \mathsf{nat}), l \mapsto l$ stencil s_0 $\quad \tau := (l_1 : \mathsf{nat}, a_2 : \mathsf{nat}), l \mapsto l_1$ stencil s_S $\quad \tau := (l_1, l_2, a_3 : \mathsf{nat}), l \mapsto l_1$ stencil s_1

Stencil s_S represents heaps that are defined on at least two locations and have (**inr** •) stored in l, e.g., the first three heaps below are valid extensions whereas the right-most is not.



A stencil is said to be *connected* if every location can be reached by traversing the heaplet starting from a public location. In the above example, stencils s_0, s_S are connected, whereas s_1 is not because a_3 is not reachable from l_1.

For a finite set of templates Σ, we call a Σ-indexed family of stencils over a public world w a *cover*. We refer to *connected covers* if all its stencils are connected, *non-overlapping covers* if the collections of heaps represented by the stencils are disjoint and *complete covers* if every heap is included by exactly one stencil in the cover. Completeness of a cover implies that it is both connected and non-overlapping.

Example 2.7. The cover $\langle s_0, s_S \rangle$ over public world $(l : \mathsf{nat})$ is complete since every heap η with a public location $(l : \mathsf{nat})$ will store in it either **inl** () (covered by s_0) or **inr** l_2 (covered by s_S). On the other hand, the cover $\langle s_0, s_S, s_1 \rangle$, is not complete since it is neither connected (due to s_1) nor is it non-overlapping: heap η_2 from Example 2.5 is included by both s_S and s_1.

A complete cover is thus a finite partition of the heap space over a common set of public locations. § 3 introduces templates formally and in § 4.1 we assemble them to form stencils and covers.

Resultlets are a means to assign stateful values to each stencil in a cover. Since stencils represent collections of heaps via the location variables, we similarly introduce variables to stateful values and talk about collections of them.

An $(X \in \mathbf{W})$-valued resultlet over template τ is thought of as a stateful value that has access to variables $\lceil \tau \rceil$. Specifically, a resultlet $inj_{\lfloor \tau \rfloor \xrightarrow{\sigma'} w'} \langle x', \eta' \rangle \in \mathsf{Res} X \tau$ has the following data:

- An injection $\lfloor \tau \rfloor \xrightarrow{\sigma'} w' \in \mathbb{W}$ representing an allocation of $w' \ominus \sigma'$ locations.
- An X-valued return value $x' : X(w' \oplus \lceil \tau \rceil)$ that can also return variables $\lceil \tau \rceil$.
- A heaplet $\eta' : \underline{\mathbb{H}}(w', w' \oplus \lceil \tau \rceil)$ that is defined on w' and has access to variables $\lceil \tau \rceil$.

Notice that resultlets cannot instantiate variables $\lceil \tau \rceil$ in their data: these variables represent the state of the heap that is left unchanged. Just as with stencils, resultlets are thought of as collections of stateful values given by all the instantiations of $\lceil \tau \rceil$.

Example 2.8. We create two $[\![\mathsf{ref}_{\mathsf{nat}}]\!]$-valued resultlets over templates $(l : \mathsf{nat})$ and $(l_1, a_2 : \mathsf{nat})$. Resultlet r_c allocates a new cell l', creates a cycle with l and returns l, whereas r_S does not allocate any new cells but instead only writes **inr** a_2 to l_1.



$r_c := inj_{(l:\mathsf{nat}) \le (l,l':\mathsf{nat})} \langle l, \eta_c \rangle \qquad r_S := inj_{(l_1,a_2:\mathsf{nat}) \le (l_1,a_2:\mathsf{nat})} \langle l_1, \eta_S \rangle$

Recall the four stateful values from Example 2.5. Resultlet r_S can be extended to the right-most three using data $l_2\,\boxed{\textbf{inl}\,()}$ and $l_3\,\boxed{\textbf{inl}\,()}$ $l_2\,\boxed{\textbf{inr}\,\bullet}$. In contrast, r_c cannot be extended to the left-most stateful value since l_2 is an existing location whereas l' has been allocated.

For a finite set of templates Σ, we call a Σ-indexed family of X-valued resultlets an *evaluation*. Resultlets and evaluations are further described in § 4.2.

Finitary Computations. We link covers with evaluations to describe the finitary FGLS computations. Every triple consisting of a finite set of templates Σ, a Σ-indexed complete cover over w and Σ-indexed X-valued evaluation describes a FGLS computation in $\mathcal{T}Xw$ in a non-unique way.

For instance, $[(l : \mathrm{nat}) \mapsto s_0, r_c; (l_1, a_2 : \mathrm{nat}) \mapsto s_S, r_S\,]$ represents a finitary FGLS computation over public world $(l : \mathrm{nat})$ and templates $(l : \mathrm{nat})$ and $(l_1, a_2 : \mathrm{nat})$. The computation that it realises is defined at $(l : \mathrm{nat}) \xrightarrow{\sigma} w', \eta' : \mathbb{H}(w')$, by case splitting on the contents of cell $\pi_{\sigma(l)}\eta'$. If it is $\mathbf{inl}\,()$, it overwrites it with the cycle given by r_c leaving the rest of the heap unchanged. If it is $\mathbf{inr}\,l_2$, for some location $l_2 \in w'$, it leaves the whole heap η' unchanged. In both cases $\sigma(l)$ is returned.

We say that $\phi \in \mathcal{T}Xw$ is finitary if it can be expressed using covers and evaluations. In § 4.3 we show that the finitary computations are closed under the functor and monad structure and therefore define a submonad of $\mathcal{T}$.

3 Worlds, Templates and Instantiations

In this section we introduce the machinery needed to describe the finitary FGLS monad. We first revisit worlds and instantiations (§ 3.1) from [15] and then extend worlds to two-kinded worlds called *templates* (§ 3.2). We then introduce instantiation data to templates (§ 3.3) and lastly develop the theory for collections of templates with instantiation data (§ 3.4).

3.1 Worlds and Instantiations

We first revisit categories $\mathbb{W}$ and $\mathbb{WI}$ introduced in § 2.2.

Recall that category $\mathbb{W}$ has worlds w as objects and morphisms are sort-preserving injections $w \xrightarrow{\sigma} w'$. We can form the disjoint union of two worlds denoted by $w_1 \oplus w_2$ that has $|w_1| + |w_2|$ many locations. This construction extends to a functor $\oplus : \mathbb{W} \times \mathbb{W} \to \mathbb{W}$ together with injections $i_1 : w_1 \to w_1 \oplus w_2$ and $i_2 : w_2 \to w_1 \oplus w_2$ but note that it does not satisfy the universal property of coproducts because of injectivity of morphisms in $\mathbb{W}$.

We equip $\mathbb{W}$ with instantiation data to form category $\mathbb{WI}$, which has worlds w as objects, and morphisms $w \xrightarrow{\varepsilon} w'$ consist of a sort preserving injection $u\varepsilon$ and heaplet

$$\eta_\varepsilon : \underline{\mathbb{H}}(w' \ominus u\varepsilon, w') = \prod_{l:C \in w' \ominus u\varepsilon} [\![ctype\, C]\!](w')$$

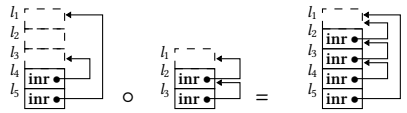
Semantic full-ground values $[\![ctype\, C]\!]$ are given for reference types using the hom-functor $[\![\mathrm{ref}_C]\!] = \mathbb{W}((l : C), -)$, and thus $\sigma \in [\![\mathrm{ref}_C]\!](w)$ is a morphism $(l : C) \xrightarrow{\sigma} w$ in $\mathbb{W}$, i.e., a selection of a single C-sorted location in w. The $\times$, $+$ types are given via the cartesian closed structure of $\mathbf{W}$. Note that semantic values are in one-to-one correspondence with syntactic closed full ground values: $[\![ctype\, C]\!](w) \cong \cdot \vdash^{\mathrm{v}}_w : ctype\, C$.

Composition of $w_1 \xrightarrow{\varepsilon_1} w_2 \xrightarrow{\varepsilon_2} w_3$ is defined by composition of the underlying injection and composition of the instantiation data $\eta_{\varepsilon_2 \circ \varepsilon_1} : \underline{\mathbb{H}}(w_3 \ominus (u\varepsilon_2 \circ u\varepsilon_1), w_3)$ is given by

$$l : C \in w_3 \ominus (u\varepsilon_2 \circ u\varepsilon_1) \mapsto \begin{cases} [\![ctype\, C]\!](u\varepsilon_2)(\pi_{l'}\eta_{\varepsilon_1}) & \text{if } u\varepsilon_2(l') = l \\ \pi_l \eta_{\varepsilon_2} & \text{if } l \in w_3 \ominus u\varepsilon_2 \end{cases}$$

A $l : C \in w_3 \ominus (u\varepsilon_2 \circ u\varepsilon_1)$ comes either from $w_2 \ominus u\varepsilon_1$ or from $w_3 \ominus u\varepsilon_2$, and $[\![ctype\ C]\!](u\varepsilon_2)$ promotes locations w_2 to w_3.

Example 3.1. The composition of $l_1 : \mathsf{nat} \overset{\varepsilon_1}{\hookrightarrow} (l_1, l_2, l_3 : \mathsf{nat}) \overset{\varepsilon_2}{\hookrightarrow} (l_1, l_2, l_3, l_4, l_5 : \mathsf{nat})$ is given as



heap η_{ε_2} heap η_{ε_1} heap $\eta_{\varepsilon_2 \circ \varepsilon_1}$

$\mathbb{W}$ has pullbacks $\begin{array}{ccc} w_1 \otimes_w w_2 & \xrightarrow{\pi_2} & w_2 \\ {\scriptstyle \pi_1}\downarrow & \lrcorner & \downarrow{\scriptstyle \sigma_2} \\ w_1 & \xrightarrow[\sigma_1]{} & w \end{array}$ given by image intersection: $\underline{w_1 \otimes_w w_2} = \mathsf{Img}(\sigma_1) \cap \mathsf{Img}(\sigma_2)$.

3.2 Templates

We introduce worlds with two kinds of locations (i) concrete locations that represent pointers to allocated data and (ii) location variables that stand for extension points that hold *arbitrary cell contents*. These two-kinded worlds, called *templates*, are modelled using category $\mathbb{T}$ given by

- *Objects:* A world w^+ with a chosen subset $w^- \hookrightarrow w^+$.
- *Morphisms:* A morphism $(w^- \hookrightarrow w^+) \overset{\rho}{\to} (\hat{w}^- \hookrightarrow \hat{w}^+)$ is a sort-preserving injection $w^+ \overset{\rho^+}{\longrightarrow} \hat{w}^+ \in \mathbb{W}$ that preserves the chosen subset, that is, the restriction of ρ^+ to w^- factors through $\hat{w}^-$: $w^- \overset{\exists}{\dashrightarrow} \hat{w}^- \hookrightarrow \hat{w}^+$, with $\rho^+|_{w^-} : w^- \to \hat{w}^+$.

Composition is given by composition of the underlying injection. We have two forgetful functors $\lfloor - \rfloor, \lceil - \rceil : \mathbb{T} \to \mathbb{W}$, projecting the $-$ and $+$ components respectively: $\lfloor w^- \hookrightarrow w^+ \rfloor = w^-$, $\lceil w^- \hookrightarrow w^+ \rceil = w^+$. We use metavariable τ to range over objects (templates) in $\mathbb{T}$, and call $\lfloor \tau \rfloor$ its concrete locations, $\lceil \tau \rceil$ its combined locations, and $\lceil \tau \rceil := \lceil \tau \rceil \setminus \lfloor \tau \rfloor$ its location variables. If it is clear from context, we overload $\lfloor \tau \rfloor := \lfloor \tau \rfloor \hookrightarrow \lceil \tau \rceil$ and $\lceil \tau \rceil := \lceil \tau \rceil \hookrightarrow \lceil \tau \rceil$ for the inclusions into $\lceil \tau \rceil$. Let 1_w be the template with only w concrete locations, and let $?^w$ represent the template with only w location variables.

Every morphism $\rho : \tau \to \hat{\tau}$ induces a morphism $\lceil \rho \rceil := \lceil \tau \rceil \xrightarrow{\lceil \tau \rceil} \lceil \tau \rceil \xrightarrow{\lceil \rho \rceil} \lceil \hat{\tau} \rceil$ in $\mathbb{W}$ and every morphism in $\mathbb{T}$ is mono, and epis are exactly the morphisms that have a surjective $\lceil \rho \rceil$ component. Moreover, concrete locations $\lfloor \tau \rfloor$ cannot be mapped to variables $\lceil \hat{\tau} \rceil$ by ρ and so concrete locations cannot become uninstantiated.

We select specific locations by intersecting the images, e.g., $\lfloor \hat{\tau} \rfloor \otimes_{\lceil \hat{\tau} \rceil} \lceil \tau \rceil := \mathsf{Img}(\lfloor \hat{\tau} \rfloor) \cap \mathsf{Img}(\lceil \rho \rceil)$, refers to the variables $\lceil \tau \rceil$ that are instantiated by ρ and similarly $\lceil \hat{\tau} \rceil \otimes_{\lceil \hat{\tau} \rceil} (\lceil \hat{\tau} \rceil \ominus \lceil \rho \rceil)$ represents variables $\lceil \hat{\tau} \rceil$ that are fresh w.r.t. ρ ($\lceil \hat{\tau} \rceil \ominus \lceil \rho \rceil$ are the new locations introduced by ρ).

Let l range over concrete locations, and let a, b range over location variables. We write templates as single worlds $l_1 : C_1, a_2 : C_2, l_3 : C_3$, to really mean the template given by the inclusion $l_1 : C_1, l_3 : C_3 \hookrightarrow l_1 : C_1, a_2 : C_2, l_3 : C_3$.

Example 3.2. Consider the following morphism in $\mathbb{T}$:

$$l_1 : C_1, a_2 : C_2 \xrightarrow{l_1 \mapsto l_1, a_2 \mapsto l_2} l_1 : C_1, l_2 : C_2, a_3 : C_3$$

It instantiates variable a_2 from its domain, and introduces a new fresh variable a_3.

The category $\mathbb{T}$ has initial object $?^{\emptyset}$, and equalizers and pullbacks such that $\lfloor - \rfloor, \lceil - \rceil$ preserve these. It is also equivalent to the arrow category $\mathsf{Arr}(\mathbb{W})$.

3.2.1 Local Independent Coproducts. For a span $\tau_1 \xleftarrow{\rho_1} \tau \xrightarrow{\rho_2} \tau_2$ its local independent coproduct gives the least way to close the span into a commuting square, that is, it makes the least assumptions on which locations from τ_1 and τ_2 are related (at both the $\lfloor - \rfloor$ and $[-]$ level). We define its local independent coproduct $\tau_1 \oplus_\tau \tau_2$ on the $[-]$ level as

$$\begin{array}{ccc} \tau & \xrightarrow{\rho_2} & \tau_2 \\ \downarrow{\scriptstyle \rho_1} & & \downarrow{\scriptstyle \rho_2^* \rho_1} \\ \tau_1 & \xrightarrow{\rho_1^* \rho_2} & \tau_1 \oplus_\tau \tau_2 \end{array} \qquad \begin{array}{ccc} [\tau] & \xrightarrow{[\rho_1]} & [\tau_1] \\ {\scriptstyle [\rho_2]}\downarrow & & \downarrow{\scriptstyle [\rho_1]^*[\rho_2]} \\ [\tau_2] & \xrightarrow[[\rho_2]^*[\rho_1]]{} & [\tau] \oplus ([\tau_1] \ominus [\rho_1]) \oplus ([\tau_2] \ominus [\rho_2]) =: [\tau_1 \oplus_\tau \tau_2] \end{array}$$

$[\rho_1]^*[\rho_2]$, $[\rho_2]^*[\rho_1]$ are given by the canonical isomorphism of injections: $[\tau_i] \cong [\tau] \oplus ([\tau_i] \ominus [\rho_i])$.

The concrete locations $\lfloor \tau_1 \oplus_\tau \tau_2 \rfloor$ are

$$\lfloor \tau_1 \oplus_\tau \tau_2 \rfloor := \mathsf{Img}([\rho_1]^*[\rho_2] \circ \lfloor \tau_1 \rfloor) \cap \mathsf{Img}([\rho_2]^*[\rho_1] \circ \lfloor \tau_2 \rfloor)$$

Thus we only relate locations $[\tau]$ from $[\tau_1]$ and $[\tau_2]$ and a location $l : C \in [\tau_1 \oplus_\tau \tau_2]$ is concrete iff it is concrete in either $\lfloor \tau_1 \rfloor$ or $\lfloor \tau_2 \rfloor$.

Example 3.3. $\begin{array}{ccc} a_1 : \mathsf{nat} & \hookrightarrow & a_1, l_2 : \mathsf{nat} \\ {\scriptstyle a_1 \mapsto l_1}\downarrow & & \downarrow{\scriptstyle a_1 \mapsto l_1} \\ l_1, a_3 : \mathsf{nat} & \hookrightarrow & l_1, l_2, a_3 : \mathsf{nat} \end{array}$ is a local independent coproduct square since a_3 and l_2 are not conflated and a_3 is not instantiated.

Local independent coproducts can be given a universal property using the independence structures of [30]. A commuting square is called *independent* if its $[-]$-image is a pullback, and a local independent coproduct square is the universal independent square, that is, every other independent square factors uniquely through it.

3.2.2 Variable Preserving Morphisms. We call morphisms in $\mathbb{T}$ that do not instantiate location variables from their domain as *variable preserving*.

Definition 3.4. A morphism $\rho : \tau_1 \to \tau_2$ is variable preserving if it satisfies $\lfloor \tau_2 \rfloor \otimes_{[\tau_2]} \lceil \tau_1 \rceil = \emptyset$.

Example 3.5. $a_1 : C_1 \hookrightarrow a_1 : C_1, a_2 : C_2$ is variable preserving since a_1 remains a variable in the codomain, whereas $a : C \xrightarrow{a \mapsto l} l : C$ is not.

Variable preserving morphisms are closed under composition, and characterise exactly the regular monos in $\mathbb{T}$.

Lemma 3.6. Local independent coproducts reflect variable preserving morphisms, that is, in a span $\tau_1 \xleftarrow{\rho_1} \tau \xrightarrow{\rho_2} \tau_2$, if ρ_2 is variable preserving, then so is $\tau_1 \xrightarrow{\rho_1^* \rho_2} \tau_1 \oplus_\tau \tau_2$.

3.2.3 Variable Reflecting Morphisms. Similarly, we call a morphism that does not introduce location variables as *variable reflecting*.

Definition 3.7. A $\rho : \tau_1 \to \tau_2$ is variable reflecting if it satisfies $([\tau_2] \ominus \lceil \rho \rceil) \otimes_{[\tau_2]} \lceil \tau_2 \rceil = \emptyset$.

Example 3.8. $a : C \xrightarrow{a \mapsto l} l : C$ is a variable reflecting whereas $a_1 : C_1 \hookrightarrow a_1 : C_1, a_2 : C_1$ is not.

Variable reflecting morphisms are closed under composition and local independent coproducts reflect them in the same manner as Lemma 3.6.

3.3 Template Instantiations

We extend templates and their morphisms with instantiation data. Let $\mathbb{TI}$ be the category with objects being templates $\tau \in \mathrm{Obj}(\mathbb{T})$, and morphisms $\tau_1 \xrightarrow{\varepsilon} \tau_2 \in \mathbb{TI}$ are pairs consisting of a template morphism $\tau_1 \xrightarrow{u\varepsilon} \tau_2 \in \mathbb{T}$ and instantiation data $\eta_\varepsilon : \underline{\mathbb{H}}(\lfloor\tau_2\rfloor \ominus \lfloor u\varepsilon\rfloor, [\tau_2])$. η_ε defines a value that has access to all $[\tau_2]$ for every $l : C \in \lfloor\tau_2\rfloor$ that is not hit by $\lfloor u\varepsilon\rfloor$.

Composition of $\tau_1 \xrightarrow{\varepsilon_1} \tau_2 \xrightarrow{\varepsilon_2} \tau_3$ is defined by composition in $\mathbb{T}$ for the underlying injection and composition of the instantiation data $\eta_{\varepsilon_2 \circ \varepsilon_1} : \underline{\mathbb{H}}(\lfloor\tau_3\rfloor \ominus (\lfloor\varepsilon_2\rfloor \circ \lfloor\varepsilon_1\rfloor), [\tau_3])$ is given as in $\mathbb{WI}$:

$$l : C \mapsto \begin{cases} [\![ctype\, C]\!][\varepsilon_2](\pi_{l'}\eta_{\varepsilon_1}) & \text{if } \lfloor\varepsilon_2\rfloor(l') = l \\ \pi_l \eta_{\varepsilon_2} & \text{if } l \in \lfloor\tau_3\rfloor \ominus \lfloor\varepsilon_2\rfloor \end{cases}$$

We have a forgetful functor $u : \mathbb{TI} \to \mathbb{T}$, lifted functors $\lfloor-\rfloor, [-] : \mathbb{TI} \to \mathbb{W}$, and functor $1_- : \mathbb{WI} \hookrightarrow \mathbb{TI}$ which maps to concrete templates $w \mapsto 1_w$. We further inherit the terminology of $\mathbb{T}$ and talk about concrete locations and location variables, and talk about variable preserving and reflecting morphisms by noticing that u preserves and reflects these properties. The forgetful functor preserves equalizers and pullbacks.

Definition 3.9. The template heap functor is defined as $\mathbb{TH} := \mathbb{TI}(?^{\emptyset}, -) : \mathbb{TI} \to \mathrm{Set}$.

We have $\mathbb{TH}(\tau) \cong \underline{\mathbb{H}}(\lfloor\tau\rfloor, [\tau])$ and so a template heap $\eta \in \mathbb{TH}(\tau)$ defines a value for every concrete location in $\lfloor\tau\rfloor$ that has access to all locations $[\tau]$.

3.3.1 Reachability. Instantiation data introduce a notion of (un)reachable locations, refering to whether a location can be reached from a concrete location by traversing the data.

Definition 3.10. The reachable locations from $w \xrightarrow{\sigma} [\tau]$ w.r.t. a template heap $\eta : \mathbb{TH}(\tau)$ is the least factoring of σ as $w \xrightarrow{\exists\sigma_{\mathrm{r}}} w_{\mathrm{r}} \xrightarrow{\exists\sigma_{\mathrm{ur}}} [\tau]$ such that for every reachable concrete location $b : C \in w_{\mathrm{r}} \otimes_{[\tau]} \lfloor\tau\rfloor$, the locations in $\pi_b\eta$ are included in w_{r}: there exists a $V \in [\![ctype\, C]\!](w_{\mathrm{r}})$ s.t. $[\![ctype\, C]\!](\sigma_{\mathrm{ur}})(V) = \pi_b\eta$. We refer to w_{r} as the reachable and $[\tau] \ominus \sigma_{\mathrm{ur}}$ as the unreachable locations.

Example 3.11. The reachable locations from $l_1 : \mathrm{nat} \hookrightarrow (l_1, a_2, l_3 : \mathrm{nat})$ w.r.t. template heap 

are $l_1 : \mathrm{nat} \hookrightarrow (l_1, a_2 : \mathrm{nat}) \hookrightarrow (l_1, a_2, l_3 : \mathrm{nat})$, since l_3 is not reachable from l_1.

Lemma 3.12. For every $w \xrightarrow{\sigma} [\tau]$ and $\eta : \mathbb{TH}(\tau)$ the reachable location factoring $\sigma_{\mathrm{r}}, \sigma_{\mathrm{ur}}$ exists.

Proof Sketch. World w^{r} is given by the image $\mathrm{Img}(\sigma)$ and all locations encountered when traversing η starting from every concrete location that is in w (i.e., $w \otimes_{[\tau]} \lfloor\tau\rfloor$).

3.3.2 Connectivity. Using reachability we identify *connected* morphisms in $\mathbb{TI}$ with the property that all locations in the codomain are reachable from locations in the domain. We define connectivity formally by giving a factorisation of morphisms in $\mathbb{TI}$ that splits its reachable and unreachable components.

Definition 3.13 (Factorisation in $\mathbb{TI}$)**.** For a morphism $\varepsilon : \tau_1 \to \tau_2$, we consider the reachable locations from $[\varepsilon]$ w.r.t. $\eta_\varepsilon : \mathbb{TH}(\lfloor\tau_2\rfloor \ominus \lfloor\varepsilon\rfloor \xrightarrow{\lfloor\varepsilon\rfloor^{\mathrm{c}}} \lfloor\tau_2\rfloor \xrightarrow{\lfloor\tau_2\rfloor} [\tau_2])$. Let $[\tau_1] \xrightarrow{[\varepsilon]^{\mathrm{r}}} w^{\mathrm{r}} \xrightarrow{[\varepsilon]^{\mathrm{ur}}} [\tau_2]$ be the factoring of ε with the chosen subset for w^{r} being the reachable concrete locations $w^{\mathrm{r}} \otimes_{[\tau_2]} \lfloor\tau_2\rfloor$. The property of Definition 3.10 allows us to split η_ε into data for these two morphisms.

For a morphism $\varepsilon : \tau_1 \to \tau_2$, we write $\mathrm{c}(\varepsilon) : \tau_1 \to \tau_{\mathrm{c}(\varepsilon)}$ and $\mathrm{uc}(\varepsilon) : \tau_{\mathrm{c}(\varepsilon)} \to \tau_2$ for its factorisation given by Definition 3.13.

Example 3.14. The factoring of $(a_1 : \mathsf{nat}) \xrightarrow{a_1 \mapsto l_1} (l_1, a_2, l_3 : \mathsf{nat})$ with data  is

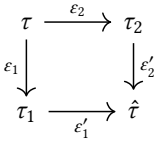
$$(a_1 : \mathsf{nat}) \xrightarrow{a_1 \mapsto l_1} (l_1, a_2 : \mathsf{nat}) \hookrightarrow (l_1, a_2, l_3 : \mathsf{nat}) \text{ with data } \text{  } \text{ and } \text{  }$$

A morphism $\varepsilon : \tau_1 \to \tau_2$ is said to be connected if $\mathsf{uc}(\varepsilon)$ is an isomorphism and correspondingly, it is disconnected if $\mathsf{c}(\varepsilon)$ is iso. We have that $\mathsf{c}(\varepsilon)$ is itself connected, and $\mathsf{uc}(\varepsilon)$ is disconnected.

Epimorphisms in $\mathbb{TI}$ are exactly the connected morphisms, and regular monomorphisms are exactly the disconnected morphisms. Disconnected morphisms are alternatively characterised as the variable preserving ones. We use arrows $\twoheadrightarrow$ for epis, and $\rightarrowtail_{\mathsf{r}}$ for regular monos.

Lemma 3.15 (Orthogonal Factorisation System in $\mathbb{TI}$)**.** The classes of epis (connected) and regular monos (disconnected) morphisms gives rise to an orthogonal factorisation system in $\mathbb{TI}$.

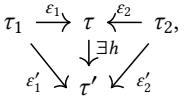
3.3.3 Unification. Dealing with concrete cells that can store location variables naturally leads us to unification that answers the question of whether two template heaps with a common set of starting locations unify to a larger template heap that includes them both. In the case of $\mathbb{TI}$, template heaps are morphisms and so we phrase unification as follows.

Definition 3.16. A span $\tau_1 \xleftarrow{\varepsilon_1} \tau \xrightarrow{\varepsilon_2} \tau_2$ unifies if there exists a commuting square 

We call $\varepsilon_1', \varepsilon_2'$ a unifier of $\varepsilon_1, \varepsilon_2$.

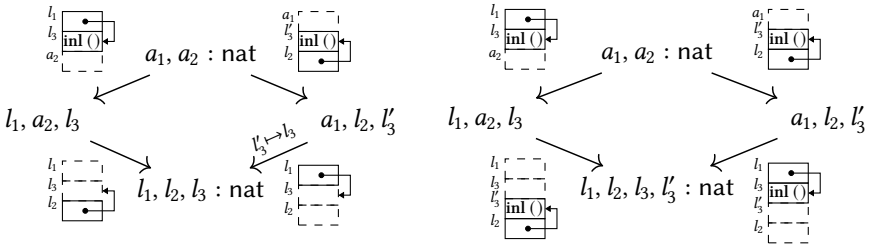
Example 3.17. Not all spans unify, e.g.,: $(l : \mathsf{bool}) \xleftarrow{l\ \boxed{\mathsf{false}}} (a : \mathsf{bool}) \xrightarrow{l\ \boxed{\mathsf{true}}} (l : \mathsf{bool})$

We consider a partial order on unifiers of the same span defined as

$$\tau_1 \xrightarrow{\varepsilon_1} \tau \xleftarrow{\varepsilon_2} \tau_2 \;\leq\; \tau_1' \xrightarrow{\varepsilon_1'} \tau' \xleftarrow{\varepsilon_2'} \tau_2' \quad \textit{iff} \quad \text{  }$$

In general, a span might not have a least unifier as the following example shows.

Example 3.18. Consider the following two unifiers of the same span with the underlying $\mathbb{T}$ morphisms given by the indices.



The resulting heaps  and  are incomparable as l_3, l_3' cannot be collapsed to a one cell.

General Unifiers. While least unifiers do not necessarily exist, there exist unifiers which we call *general* with the property that when taken together jointly serve as the least element.

Definition 3.19. A unifier $\tau_1 \xrightarrow{\varepsilon_1'} \tau' \xleftarrow{\varepsilon_2'} \tau_2$ for a span $\tau_1 \xleftarrow{\varepsilon_1} \tau \xrightarrow{\varepsilon_2} \tau_2$ is called *general* if $\lfloor \varepsilon_1' \rfloor, \lfloor \varepsilon_2' \rfloor$ are jointly surjective and $[\varepsilon_1'], [\varepsilon_2']$ are jointly surjective.

Intuitively, a general unifier unifier does not introduce new concrete locations or variables that do not come from the span, that is, $\eta_{\varepsilon_1'}$ is completely determined by η_{ε_2} and vice versa.

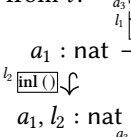
Example 3.20. Both unifiers from Example 3.18 are most general unifiers.

The jointly surjective property of general unifiers guarantees that every unifier uniquely factors through exactly one isomorphic class of general unifiers. It also implies that for a given span, there can only be a finite number of isomorphic classes of its general unifiers.

Unification and Local Independence Coproducts. Local independent coproducts in $\mathbb{T}$ (see § 3.2.1), when equipped with instantiation data are themselves general unifiers.

Definition 3.21 (Local Independent General Unifiers)**.** A unifier is called a local independent general unifier if its u image is a local independent coproduct square.

Consider a span $\tau_1 \xleftarrow{\varepsilon_1} \tau \xrightarrow{\varepsilon_2}_{\mathrm{r}} \tau_2$, such that ε_2 is variable preserving. Since ε_2 does not instantiate any variables, the data of η_{ε_2} does not have to agree with any cells in η_{ε_1} for the morphisms to commute. As a result, the local independent general unifier exists and relates locations in τ_1 and τ_2 only from ones that come from τ.

Example 3.22. The square 

from Example 3.3 equipped with data is a local independent general unifier.

3.4 Families of Template Instantiations

The last building block required to describe the finitary monad is a means to talk about collections of template heaps related by a common set of public locations. We model these collections using the $\mathsf{Fam}(\mathbb{TI})$ construction, representing finite families of templates and their instantiations:

- Objects: A finite set I, and an I-indexed set of templates $(\tau_i \in \mathsf{Obj}(\mathbb{TI}))_{i\in I}$.
- Morphisms: A $(\tau_i)_{i\in I} \xrightarrow{\phi} (\tau_j)_{j\in\mathcal{J}}$ consists of a mapping $|\phi| : \mathcal{J} \to I$ and a $\mathcal{J}$-indexed set of morphisms $(\phi_j : \tau_{|\phi|(j)} \to \tau_j)_{j\in\mathcal{J}}$

Notice that $\mathsf{Fam}(\mathbb{TI})$ is the free finite product completion of $\mathbb{TI}$. We have a forgetful functor $|-| : \mathsf{Fam}(\mathbb{TI}) \to \mathsf{Fin}^{\mathrm{op}}$ that maps into the indexing sets. Let Σ range over objects in $\mathsf{Fam}(\mathbb{TI})$ and we use list notation to write objects in $\mathsf{Fam}(\mathbb{TI})$.

Example 3.23.

$$\varepsilon_0 := \ (a : \mathsf{nat}) \xrightarrow[a\mapsto l]{l\ \boxed{\mathsf{inl}\ ()}} (l : \mathsf{nat}) \qquad \varepsilon_{\mathsf{S}} := \ (a : \mathsf{nat}) \xrightarrow[a\mapsto l_1]{\substack{a_2 \\ l_1}\ \bullet} (l_1 : \mathsf{nat}, a_2 : \mathsf{nat})$$

$\varepsilon_0, \varepsilon_{\mathsf{S}}$ give us a morphism $[a : \mathsf{nat}] \xrightarrow{\langle \varepsilon_0, \varepsilon_{\mathsf{S}} \rangle} [(l : \mathsf{nat}); (l_1 : \mathsf{nat}, a_2 : \mathsf{nat})] \in \mathsf{Fam}(\mathbb{TI})$.

Lemma 3.24. $\mathsf{Fam}(\mathbb{TI})$ has pushouts given by the general unifiers of $\mathbb{TI}$.

Proof Sketch. The pushout of $\begin{array}{ccc} \Sigma & \xrightarrow{\psi} & \Sigma_2 \\ \downarrow\phi & & \downarrow\psi^*\phi \\ \Sigma_1 & \xrightarrow[\phi^*\psi]{} & \Sigma_1 \oplus_\Sigma \Sigma_2 \end{array}$ selects the (representative) general unifiers for every pair of matching indices $\{\langle i, j\rangle \in |\Sigma_1| \times |\Sigma_2| : |\phi|(i) = |\psi|(j)\}$. That is, for every matching pair i, j, we have a finite number of representative general unifiers $\Sigma_{\mathrm{GU}(\phi_i,\psi_j)}$ with morphisms

$\tau_{|\phi|(i)} \xrightarrow{\phi_i^*\psi_j} \Sigma_{\mathrm{GU}(\phi_i,\psi_j)} \xleftarrow{\psi_j^*\phi_i} \tau_{|\psi|(j)}$ in $\mathsf{Fam}(\mathbb{TI})$. The pushout is then given by the pairing of all these morphisms

$$\Sigma_1 \oplus_\Sigma \Sigma_2 := \prod_{\langle i,j\rangle \in |\Sigma_1|\times|\Sigma_2| \,:\, |\phi|(i)=|\psi|(j)} \Sigma_{\mathrm{GU}(\phi_i,\psi_j)}$$

and $\phi^*\psi := \langle \ldots, i,j \mapsto \phi_i^*\psi_j, \ldots\rangle$, $\psi^*\phi := \langle \ldots, i,j \mapsto \phi_j^*\psi_i, \ldots\rangle$.

We further characterise collections in $\mathsf{Fam}(\mathbb{TI})$ according to how the individual morphisms in the collections interact.

Definition 3.25. A morphism $\phi : \Sigma \to \Sigma'$ is said to be *connected* if every $\phi_{j\in|\Sigma'|}$ is connected.

Example 3.26. The morphism from Example 3.23 is connected as both $\varepsilon_0, \varepsilon_S$ do not introduce unreachable locations.

Definition 3.27. A morphism $\phi : \Sigma \to \Sigma'$ is said to be *non-overlapping* if every distinct pair of morphisms with the same root, $\tau_i \xleftarrow{\phi_i} \tau_k \xrightarrow{\phi_j} \tau_j$, do not unify. That is, for every $i,j \in |\Sigma'|$ s.t. $i \neq j$:

$$|\phi|(i) = |\phi|(j) \implies \forall \tau_i \xrightarrow{\varepsilon_i} \hat{\tau}, \tau_j \xrightarrow{\varepsilon_j} \hat{\tau}.\, \varepsilon_i \circ \phi_i \neq \varepsilon_j \circ \phi_j$$

Example 3.28. The morphism from Example 3.23 is non-overlapping since any extension of ε_0 will have **inl** () at l whereas ε_S will have **inr** a_2. On the other hand, adding

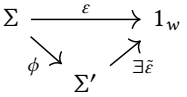
$$\varepsilon_1 := \quad (a : \mathsf{nat}) \xrightarrow[a \mapsto l_1]{} (l_1 : \mathsf{nat}, l_2 : \mathsf{nat})$$



to the collection breaks the non-overlapping property.

Epimorphisms in $\mathsf{Fam}(\mathbb{TI})$ are exactly the connected and non-overlapping morphisms.

Definition 3.29. We say that an epi $\phi : \Sigma \to \Sigma'$ is *complete* if for every morphism $\varepsilon : \Sigma \to 1_w$ there exists a $\tilde{\varepsilon} : \Sigma' \to 1_w$ such that



The epi property of ϕ guarantees that the induced morphisms are unique, and complete morphisms are closed under composition and are stable under pushouts.

Example 3.30. The collection $\langle \varepsilon_0, \varepsilon_S\rangle$ is complete: a morphism $a : \mathsf{nat} \xrightarrow{\varepsilon} 1_w$ is a heap with w-many concrete locations and a public location given by $[\varepsilon](a)$. Since $\langle \varepsilon_0, \varepsilon_S\rangle$ covers both possible inhabitants of $[\varepsilon](a)$, every such ε can be factored through either ε_0 or ε_S.

4 Finitary FGLS Monad

We now connect the combinatorial machinery developed in § 3 with the goal of describing the finitary FGLS computations. The central idea is that a finitary computation can be expressed using finitely-many stateful values with variables, each assigned to one stencil in a complete cover. We first formally define *stencils* and *covers* (§ 4.1), we then define collections of stateful values using *resultlets* (§ 4.2), and link covers and resultlets to local state computations using *realisations* in (§ 4.2). We finally present the finitary monad in (§ 4.3).

4.1 Covers and Matching

We model stencils (§ 2.4), that is collections of heaps with a common set of public locations as morphisms in $\mathbb{TI}$.

Definition 4.1 (Stencil Functor)**.**

$$\mathcal{S}_w = \mathbb{TI}(?^w, -) \cong \mathbb{W}(w, [-]) \times \mathbb{TH} : \mathbb{TI} \to \mathsf{Set}$$

with the obvious functorial action. A stencil $?^w \to \tau$ that does not have unreachable locations is said to be connected, i.e., it is a connected morphism (§ 3.3.2).

We call finite collections of stencils with a common set of public locations as *covers*.

Definition 4.2 (Cover)**.** For a finite family of templates $\Sigma \in \mathsf{Fam}(\mathbb{TI})$, and world $w \in \mathsf{Obj}(\mathbb{W})$, a *cover* cov over $\langle \Sigma, w \rangle$ is an indexed family of stencils $\prod_{\tau_i \in \Sigma} \mathcal{S}_w \tau_i$, that is, it is a morphism $\mathsf{cov} : ?^w \to \Sigma \in \mathsf{Fam}(\mathbb{TI})$.

Example 4.3. $\langle \varepsilon_0, \varepsilon_S \rangle$ from Example 3.23 is a cover over $\langle (a : \mathsf{nat}), [(l : \mathsf{nat}); (l_1 : \mathsf{nat}, a_2 : \mathsf{nat})] \rangle$.

We inherit the terminology from § 3.4, and talk about connected, non-overlapping, complete covers, and we further describe three operations on covers: (i) hiding some public locations, (ii) rebasing over a different world and (iii) intersecting two covers.

Definition 4.4 (Hiding Locations in a Cover)**.** For a renaming $\sigma : w \rightarrowtail w'$, and a cover cov over $\langle w', \Sigma \rangle$, we have a cover over $\langle w, \Sigma \rangle$ by precomposition

$$\mathsf{hide}_\sigma(\mathsf{cov}) := ?^w \xrightarrow{?^\sigma} ?^{w'} \xrightarrow{\mathsf{cov}} \Sigma$$

Hiding does not preserve connectivity, non-overlapping, or completeness in general.

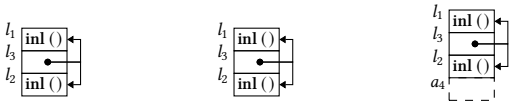
Definition 4.5 (Rebasing a cover)**.** For a $\sigma : w \rightarrowtail w'$ and cover cov over $\langle w, \Sigma \rangle$, we rebase cov to a cover cov^σ over $\langle w', \Sigma^\sigma \rangle$ using the pushout $\begin{array}{ccc} ?^w & \xrightarrow{?^\sigma} & ?^{w'} \\ {\scriptstyle \mathsf{cov}}\downarrow & \ulcorner & \downarrow{\scriptstyle \mathsf{cov}^\sigma} \\ \Sigma & \xrightarrow[\mathsf{rebase}^\sigma]{} & \Sigma^\sigma \end{array}$ in $\mathsf{Fam}(\mathbb{TI})$ (see Lemma 3.24).

Cover rebasing preserves connectivity, epimorphic covers, and completeness.

Example 4.6. Consider a singleton cover $(a : \mathsf{nat2}) \xrightarrow{a \mapsto l_3} [(l_1, l_2 : \mathsf{nat}, l_3 : \mathsf{nat2})]$ with data 

and we rebase it over $(a : \mathsf{nat2}) \hookrightarrow (a : \mathsf{nat2}, b : \mathsf{nat})$. The resulting cover has three stencils

$$(a_1, a_2 : \mathsf{nat}) \to [(l_1, l_2 : \mathsf{nat}, l_3 : \mathsf{nat2}), (l_1, l_2 : \mathsf{nat}, l_3 : \mathsf{nat2}), (l_1, l_2, a_4 : \mathsf{nat}, l_3 : \mathsf{nat2})]$$



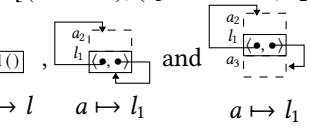
$a \mapsto l_1, b \mapsto l_1$ $\quad$ $a \mapsto l_1, b \mapsto l_2$ $\quad$ $a \mapsto l_1, b \mapsto a_4$

Definition 4.7 (Cover Intersection)**.** Given a cover cov over $\langle w, \Sigma \rangle$ and a cover cov′ over $\langle w, \Sigma' \rangle$ we define their intersection $\mathsf{cov} \otimes_w \mathsf{cov}'$ as the biggest cover that is included in both of them. It is given by the pushout $\begin{array}{ccc} ?^w & \xrightarrow{\mathsf{cov}'} & \Sigma' \\ {\scriptstyle \mathsf{cov}}\downarrow & \searrow^{\mathsf{cov} \otimes_w \mathsf{cov}'} & \downarrow{\scriptstyle \mathsf{cov}'^* \mathsf{cov}} \\ \Sigma & \xrightarrow[\mathsf{cov}^* \mathsf{cov}']{} & \Sigma \otimes_{?^w} \Sigma' \end{array}$

Finally, we use the notion of covers to describe sorts that can be finitely enumerated.

Definition 4.8 (Sort Enumeration)**.** A cover $?^{a:C} \xrightarrow{\text{cov}} \Sigma$ is called an *enumeration* of C if it is complete and every $\text{cov}_{i\in|\Sigma|} : a : C \to \tau_i$ instantiates a, i.e., $(a : C) \otimes_{\lceil\tau_i\rceil} \lceil\tau_i\rceil = \emptyset$.

We call a sort *finite* if it has an enumeration and every sort that has a full ground type interpretation as in § 2.1 is finite.

Example 4.9. The only enumeration of *ctype* zero $:= 0$ is the empty cover $(a : \text{zero}) \to \emptyset$, for *ctype* unit $:= 1$ is the singleton cover $(a : \text{unit}) \xrightarrow{a\mapsto l} [(l : \text{unit})]$ with unit data. For boolList an enumeration has stencils $(a : \text{boolList}) \to [(l : \text{bool}), (l_1 : \text{boolList}, a_2 : \text{bool}), (l_1 : \text{boolList}, a_2 : \text{bool}, a_3 : \text{boolList})]$ with data given by 

4.2 Evaluation

We call morphisms that are both variable preserving and reflecting (see § 3.2.2 and 3.2.3) as *allocation morphisms* to model that allocation only introduces new concrete locations.

Definition 4.10 (Allocation Morphisms)**.** A morphism $\rho : \tau_1 \to \tau_2 \in \mathbb{T}$ is an allocation morphism if it is both variable preserving and reflecting, and we denote it by $\to_\nu$ arrow.

For a $\rho : \tau_1 \to_\nu \tau_2$, its variables $\lceil\tau_2\rceil$ are exactly $\lceil\tau_1\rceil$, and so in particular, a morphism with a concrete template as domain $1_w \to_\nu \tau'$ implies that its codomain is also concrete: $\tau' = 1_{\lfloor\tau'\rfloor}$.

Allocation morphisms are reflected by local independent coproducts (as in Lemma 3.6), and further, identities are allocation morphisms and are closed under composition. Therefore, consider $\text{alloc}(\mathbb{T}) \subseteq \mathbb{T}$, the wide subcategory of $\mathbb{T}$ that has objects $\text{Obj}(\mathbb{T})$ and only allocation morphisms between them. Let $\tau/\text{alloc}(\mathbb{T})$ be the set of allocation morphisms with domain τ

$$\tau \xrightarrow{\rho}_\nu \hat{\tau} \in \tau/\text{alloc}(\mathbb{T}) := \coprod_{\hat{\tau}\in\text{Obj}(\mathbb{T})} \text{alloc}(\mathbb{T})(\tau, \hat{\tau})$$

Stateful Values. Recall that stateful values (§ 2.2) represent return values that live in the context of a heap. They are modelled using the monad $\mathbb{P} : \mathbf{WI} \to \mathbf{WI}$ given using a coend

$$\mathbb{P}(A)(w) = \int^{w\rightarrowtail w'\in w\downarrow u} A(w') \cong \coprod_{w\xrightarrow{\sigma}w'\in w\downarrow u} A(w')\Big/_\sim$$

$w \downarrow u$ is the comma category over world $w \in \text{Obj}(\mathbb{W})$ and $u : \mathbb{W}\mathbb{I} \to \mathbb{W}$. The quotient $/\sim$ ensures that only observable allocations are relevant which is what models garbage collection. In the setting of FGLS, we take $A = X \odot \mathbb{H}$ for some $X \in \mathbf{W}$. We refer the reader to § 2.2 for some examples.

Resultlets. Recall that collections of heaps with a common set of public locations are modelled using stencils that have location variables. We similarly generalise stateful values with location variables to describe collections of stateful values with a common heap footprint. We refer to these collections of stateful values as *resultlets*.

Definition 4.11. For a $X : \mathbb{W} \to \text{Set}$ and template τ, a resultlet r over $\langle\tau, X\rangle$ is a

$$r \in \text{Res}X\tau := \coprod_{\tau\xrightarrow{\rho}_\nu\tau'\in\text{Obj}(\tau/\text{alloc}(\mathbb{T}))} X[\tau'] \times \mathbb{T}\mathbb{H}(\tau')$$

The data of a resultlet are

- An allocation morphism $\tau \xrightarrow{\rho}_{\mathbb{V}} \tau'$ which allocates $\lfloor \tau' \rfloor \ominus \lfloor \rho \rfloor$ concrete locations.
- An X-valued return value $x : X\lceil \tau' \rceil \cong X(\lfloor \tau' \rfloor \oplus \lceil \tau \rceil)$.
- A template heap $\eta : \mathbb{TH}(\tau')$, that defines locations $\lfloor \tau' \rfloor$ that have access to both $\lfloor \tau' \rfloor, \lceil \tau \rceil$.

When τ is a concrete world 1_w, a resultlet has exactly the same data as a stateful value in the original hiding monad. Thus resultlets generalise stateful values from concrete worlds to templates with variables. Specifically, we have a map $q^X_w : \mathsf{Res}X(1_w) \to \mathbb{P}(Xu \times \mathbb{H})(w)$ for every world w:

$$inj_{1_w \xrightarrow{\rho} \tau'} \langle x, \eta \rangle \mapsto q_{w \xrightarrow{\lfloor \rho \rfloor} \lfloor \tau' \rfloor} \langle x, \eta \rangle$$

Accordingly, we lift the actions on morphisms of the monad $\mathbb{P}$ to resultlets. This lifted action is not itself functorial, since it does not account for the quotienting built into $\mathbb{P}$. Nevertheless, this is harmless: functoriality is only required at concrete worlds, and it is recovered by the map q.

Definition 4.12 (Covariant action on morphisms in **W**)**.** A $\alpha : X \Rightarrow Y$, and $\tau \in \mathrm{Obj}(\mathbb{T})$ induces $\mathsf{Res}(\alpha)_\tau : \mathsf{Res}X\tau \to \mathsf{Res}Y\tau$ defined as

$$inj_{\tau \xrightarrow{\rho'}_{\mathbb{V}} \tau'} \langle x, \eta \rangle \mapsto inj_{\rho'} \langle \alpha_{\lceil \tau' \rceil}(x), \eta \rangle$$

It changes a resultlet over $\langle \tau, X \rangle$ to one over $\langle \tau, Y \rangle$.

Definition 4.13 (Covariant action on morphisms in $\mathbb{TI}$)**.** For a $\varepsilon : \tau_1 \to \tau_2 \in \mathbb{TI}$ and a resultlet $inj_{\tau_1 \xrightarrow{\rho'}_{\mathbb{V}} \tau'} \langle x', \eta' \rangle$, we first lift ρ' to a morphism ε' in $\mathbb{TI}$ by selecting the allocation data: $\eta_{\varepsilon'} := \underline{\mathbb{H}}(\lfloor \rho' \rfloor, 1)(\eta')$. We then consider the local independent general unifier $\begin{array}{ccc} \tau_1 & \xrightarrow{\varepsilon'}_{\mathbb{V}} & \tau' \\ {\scriptstyle \varepsilon}\downarrow & & \downarrow{\scriptstyle \varepsilon'^* \varepsilon} \\ \tau_2 & \xrightarrow[\varepsilon^* \varepsilon']{}_{\mathbb{V}} & \tau_2 \oplus_{\tau_1} \tau' \end{array}$ which exists because ε' is variable preserving. Define the covariant action $\mathsf{Res}A\varepsilon : \mathsf{Res}A\tau_1 \to \mathsf{Res}A\tau_2$ as $inj_{\rho'} \langle x, \eta \rangle \mapsto inj_{u(\varepsilon^* \varepsilon')} \langle X[\varepsilon'^* \varepsilon](x), \mathbb{TH}(\varepsilon'^* \varepsilon)(\eta) \rangle$

Definition 4.14 (Contravariant action on morphisms in $\mathrm{alloc}(\mathbb{T})$)**.** Every allocation morphism $\tau_2 \xrightarrow{\rho'}_{\mathbb{V}} \tau_1$ in $\mathbb{T}$ induces $\mathsf{hide}^X_{\rho'} : \mathsf{Res}X\tau_1 \to \mathsf{Res}X\tau_2$ given by $inj_\rho \langle x, \eta \rangle \mapsto inj_{\rho \circ \rho'} \langle x, \eta \rangle$. We call this action *hiding* as it forgets some of the public locations.

We call a finite collection of resultlets as an *evaluation.*

Definition 4.15 (Evaluation)**.** For a $X : \mathbb{W} \to \mathsf{Set}$, and $\Sigma \in \mathsf{Fam}(\mathbb{TI})$, an *evaluation* is an Σ-indexed family of resultlets, $\mathsf{ev} \in \prod_{\tau_i \in \Sigma} \mathsf{Res}X\tau_i$.

In summary, resultlets play the role of compatible stateful values, and an evaluation is a finite family of resultlets, one for each stencil of a cover. The pair consisting of a cover and an evaluation can therefore be understood as a finite decision tree: the cover determines which case of the input heap we are in, and the evaluation specifies the corresponding output behaviour. Formally, we link covers and evaluations to local state computations using the notion of realisation.

Definition 4.16 (Realisation)**.** A complete cover cov over $\langle w, \Sigma \rangle$ and an X-valued evaluation ev over Σ, realises $\phi \in \mathcal{T}Xw$, notation $\mathsf{cov}, \mathsf{ev} \Vdash \phi$, if for every $\sigma : w \rightarrowtail w', \eta : \mathbb{H}(w')$,

$$(\pi_\sigma \phi)(\eta) = q^X_{w'}(\mathsf{Res}X(\tilde{\varepsilon}_\star)(\pi_{|\tilde{\varepsilon}|(\star)}\mathsf{ev}))$$

where morphism $\tilde{\varepsilon} : \Sigma \to 1_{w'}$ is given by completeness: $?^w \xrightarrow{\mathsf{cov}} \Sigma \xrightarrow{\exists \tilde{\varepsilon}} 1_{w'}$ (with $\langle \sigma, \eta \rangle$: $?^w \to 1_{w'}$)

4.3 The Monad

We are now in a position to describe the finitary fragment of the FGLS monad.

Definition 4.17 (Finitary FGLS Monad)**.**

$$\mathcal{T}_{\mathsf{fin}} : \mathbf{W} \to \mathbf{W}$$
$$\mathcal{T}_{\mathsf{fin}} Xw := \{\phi \in \mathcal{T}Xw : \exists \Sigma \in \mathsf{Fam}(\mathbb{TI}),\ \textit{complete}\ \mathsf{cov}\ \textit{over}\ \langle w, \Sigma\rangle,\ X\textit{-valued}\ \mathsf{ev}\ \textit{over}\ \Sigma\ \textit{s.t.}\ \mathsf{cov}, \mathsf{ev} \Vdash \phi\}$$

A finitary ϕ is one that can be realised by a finite family of templates, a complete cover and evaluation. The existential given here is mere existence, so the actual cover and evaluation needed to realise a finitary ϕ is not observable, nor is it unique.

The following two theorems give the functorial and monad structure of $\mathcal{T}_{\mathsf{fin}}$ by lifting it through the actions on $\mathcal{T}$. One can easily show that monad $\mathcal{T}_{\mathsf{fin}}$ is strong by a similar lifting of the strength for $\mathcal{T}$, and we therefore have a strong monad morphism $\iota : \mathcal{T}_{\mathsf{fin}} \Rightarrow \mathcal{T}$ given by the inclusion. Theorem 3 lastly shows that $\mathcal{T}_{\mathsf{fin}}$ is indeed finitary.

Theorem 1 (Functor Structure)**.** The action of $\mathcal{T}$ on morphisms $\hat{\sigma} : w \to \hat{w} \in \mathbb{W}$ and $\alpha : X \Rightarrow Y \in \mathbf{W}$ applied to finitary computations preserve finitarity.

$$\begin{array}{ccc} \mathcal{T}_{\mathsf{fin}}Xw & \overset{\exists \mathcal{T}_{\mathsf{fin}}\alpha_w}{\dashrightarrow} & \mathcal{T}_{\mathsf{fin}}Yw \\ \downarrow & & \downarrow \\ \mathcal{T}Xw & \xrightarrow{\mathcal{T}\alpha_w} & \mathcal{T}Yw \end{array} \qquad \begin{array}{ccc} \mathcal{T}_{\mathsf{fin}}Xw & \overset{\exists \mathcal{T}_{\mathsf{fin}}X\hat{\sigma}}{\dashrightarrow} & \mathcal{T}_{\mathsf{fin}}X\hat{w} \\ \downarrow & & \downarrow \\ \mathcal{T}Xw & \xrightarrow{\mathcal{T}X\hat{\sigma}} & \mathcal{T}X\hat{w} \end{array}$$

Proof Sketch. Consider a finitary $\phi \in \mathcal{T}_{\mathsf{fin}}Xw$, with Σ, cov, ev.

$\mathcal{T}\alpha_w\phi$ is given at $\sigma : w \rightarrowtail w', h : \mathbb{H}(w')$ as $\mathbb{P}(\alpha \odot \mathbb{H})_{w'}((\pi_\sigma\phi)(h))$. Then to show that $\mathcal{T}\alpha_w\phi$ is finitary take the same cover cov, and define a new evaluation ev' at $\tau_i \in \Sigma$ as $\pi_{\tau_i}\mathsf{ev}' := \mathsf{Res}(\alpha)_{\tau_i}(\pi_{\tau_i}\mathsf{ev})$.

$\mathcal{T}X\hat{\sigma}$ is given at $\sigma' : \hat{w} \rightarrowtail w', h : \mathbb{H}(w')$ as $(\pi_{\sigma' \circ \hat{\sigma}}\phi)(h)$. To show that $\mathcal{T}X\hat{\sigma}(\phi)$ is finitary, we first rebase cov using Definition 4.5 which gives a complete cover cov^σ over $\langle \hat{w}, \Sigma^\sigma\rangle$. And define evaluation ev^σ at $k \in |\Sigma^\sigma|$ as $\mathsf{Res}(X)(\mathsf{rebase}_k)(\pi_{|\mathsf{rebase}|(k)}\mathsf{ev})$.

Theorem 2 (Monad Structure)**.** The unit of $\mathcal{T}$ gives a finitary computation and the join of $\mathcal{T}$ applied to finitary computations yields a finitary one.

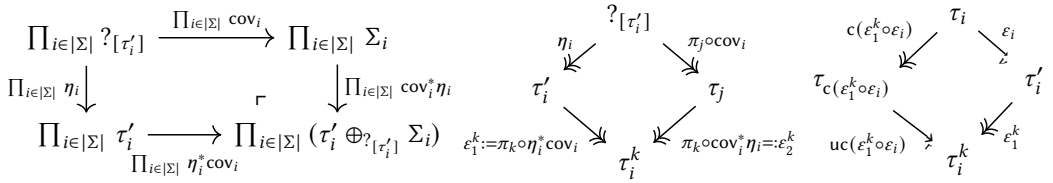
$$\begin{array}{ccc} Xw & \overset{\exists \eta^{\mathsf{fin}}_w}{\dashrightarrow} & \mathcal{T}_{\mathsf{fin}}Xw \\ & {\scriptstyle \eta_w}\searrow & \downarrow \\ & & \mathcal{T}Xw \end{array} \qquad \begin{array}{ccc} \mathcal{T}_{\mathsf{fin}}\mathcal{T}_{\mathsf{fin}}Xw & \overset{\exists \mu^{\mathsf{fin}}_w}{\dashrightarrow} & \mathcal{T}_{\mathsf{fin}}Xw \\ \downarrow & & \downarrow \\ \mathcal{T}\mathcal{T}_{\mathsf{fin}}Xw & & \\ {\scriptstyle \mathcal{T}(\hookrightarrow)_w}\downarrow & & \\ \mathcal{T}\mathcal{T}Xw & \xrightarrow{\mu_w} & \mathcal{T}Xw \end{array}$$

Proof Sketch. The unit $\eta_w(x) : \mathcal{T}_{\mathsf{fin}}Xw$ is given at $\sigma : w \to w', h : \mathbb{H}(w')$ as $q_{1_{w'}}\langle X(\sigma)(x), h\rangle$. To show that $\eta_w(x)$ is finitary, consider the identity cover $?^w \overset{1}{\Rightarrow} ?^w$. It is easy to see that it is complete. Define the evaluation at template $?^w$ as resultlet $\mathit{inj}_{?^w} \overset{1}{\Rightarrow}_{?^w}\langle x, \emptyset\rangle$.

Multiplication $\mu_w(\mathcal{T}(\hookrightarrow)_w\phi)$ involves a number of steps. From finitariness of ϕ, we have a complete cover cov over $\langle w, \Sigma\rangle$ and $(\mathcal{T}_{\mathsf{fin}}X)$-valued evaluation ev s.t. cov, ev $\Vdash \phi$. Unpacking this, the resultlet at every $\tau_i \in \Sigma$ gives

- Morphism $\rho_i : \tau_i \rightarrow_\nu \tau_i'$, finitary computation $\phi_i : \mathcal{T}_{\mathsf{fin}}X[\tau_i']$, and heap $\eta_i : \mathbb{TH}\tau_i'$.
- Every ϕ_i further gives a $\Sigma_i \in \mathsf{Fam}(\mathbb{TI})$, complete cover cov_i over $\langle[\tau_i'], \Sigma_i\rangle$ and X-valued evaluation ev_i over Σ_i.

Firstly, we define its family of templates $\Sigma^{\mu\phi}$ by unifying every $\eta_i : ?^{[\tau_i']} \to \tau_i'$ with cov_i, which is given by the following pushout in $\mathsf{Fam}(\mathbb{TI})$. We then select only templates $\tau_i^k \in \prod_{i\in|\Sigma|}(\tau_i' \oplus_{?_{[\tau_i']}} \Sigma_i)$ that are compatible with allocation ε_i (ρ_i with the allocation data): when the right-most square is a local independent general unifier.



$\Sigma^{\mu\phi}$ selects the $\tau_{\mathsf{c}(\varepsilon_1^k \circ \varepsilon_i)}$ from every such compatible τ_i^k and morphism $\Sigma \xrightarrow{\mathsf{embed}} \Sigma^{\mu\phi}$ given by the $\mathsf{c}(\varepsilon_1^k \circ \varepsilon_i)$ is complete. This defines the cover $\mathsf{cov}^{\mu\phi} = \mathsf{embed} \circ \mathsf{cov}$ since connected morphisms are closed under composition. Lastly, the evaluation $\mathsf{ev}^{\mu\phi}$ is given at every $\tau_{\mathsf{c}(\varepsilon_1^k \circ \varepsilon_i)}$ as

$$\pi_{\tau_{\mathsf{c}(\varepsilon_1^k \circ \varepsilon_i)}} \mathsf{ev}^{\mu\phi} := \mathsf{hide}_{\mathsf{uc}(\varepsilon_1^k \circ \varepsilon_i)}(\mathsf{Res}(X)(\varepsilon_2^k)(\pi_j \mathsf{ev}_i))$$

Use the hiding algebras of $\mathbb{P}$ ([15]) to prove realisation.

Theorem 3 (Finitarity)**.** Monad $\mathcal{T}_{\mathsf{fin}}$ is finitary.

Proof Sketch. We need to show that $\mathcal{T}_{\mathsf{fin}}$ preserves filtered colimits. Let $F : \mathcal{I} \to \mathbf{W}$ be a filtered colimit and we need to show the induced cocone morphism $\mathit{colimit}\,(\mathcal{T}_{\mathsf{fin}} \circ F) \dashrightarrow \mathcal{T}_{\mathsf{fin}}(\mathit{colimit}\, F)$ is iso. We give the construction of the inverse. By finitarity of $\mathcal{T}_{\mathsf{fin}}$, we have that a finitary $\phi : \mathcal{T}_{\mathsf{fin}}(\mathit{colimit}\, F)(w)$ has a finite family of objects in $\mathcal{I}$, $R \in \prod_{i\in|\Sigma_\phi|} \mathsf{Obj}(\mathcal{I})$. By filteredness, we can take the upper bound of these objects, denote it by k, with morphisms $Ri \xrightarrow{f_i} k$ for every $i \in |\Sigma_\phi|$. We then extend every resultlet $\pi_\tau \mathsf{ev}_\phi := \mathit{inj}_{\rho:\tau\to\tau'}\langle [x], \eta\rangle$ to $\mathit{inj}_\rho \langle F_{[\tau']}(f_i)(x), \eta\rangle$. We can show that this map (i) is invariant under which cover, evaluation pair that realises ϕ, and (ii) is the inverse morphism from the colimiting cocone.

5 Finitary Semantics for FGLS

In this section we give semantics to λ_{ref} from § 2.1 using the finitary FGLS monad. We interpret types as functors $[\![A]\!] : \mathbb{W} \to \mathsf{Set}$ that carry a covariant structure. We show that generic effects *read*, *write* and *alloc* are finitary computations allowing us to interpret λ_{ref} terms. Lastly, we show a correspondence between the semantics for λ_{ref} given via the two monads.

5.1 Semantics of Types

Figure 5 gives the interpretation of λ_{ref} types as covariant functors $[\![A]\!] : \mathbb{W} \to \mathsf{Set}$: given a $\sigma : w \to w'$, we can extend a $a \in [\![A]\!]w$ to a $[\![A]\!](\sigma)(a) \in [\![A]\!](w')$. For $0, 1, \times, +, \to$ the interpretation is standard using the bi-cartesian closed structure of $\mathbf{W}$ and monad $\mathcal{T}_{\mathsf{fin}}$ [23, 24]. For reference types $[\![\mathsf{ref}_C]\!]$, we use the representable functor $\mathbb{W}(l : C, -)$. Thus a $\sigma \in [\![\mathsf{ref}_C]\!](w)$ is an injection $l : C \xrightarrow{\sigma} w$ giving us a single location in w. The covariant structure of $[\![\mathsf{ref}_C]\!]$ says that location l remains available for every extension of w.

5.2 State Manipulating Generic Effects

Generic Effect Read. For every sort $C \in \mathsf{Sort}$, monad $\mathcal{T}$ supports generic effect $\mathit{read}_C : [\![\mathsf{ref}_C]\!] \to \mathcal{T}[\![\mathit{ctype}\, C]\!]$ which returns the contents of location l:

$$\mathit{read}_C(l)(w \xrightarrow{\sigma} w')(\eta : \mathbb{H}w') = q_{w' \xrightarrow{1} w'}\langle \pi_{\sigma(l)}\eta, \eta\rangle$$

$$\begin{array}{rclcrcl} [\![0]\!] & = & \mathbb{0} & & [\![A \times B]\!] & = & [\![A]\!] \times [\![B]\!] \\ [\![1]\!] & = & \mathbb{1} & & [\![A + B]\!] & = & [\![A]\!] + [\![B]\!] \\ [\![\mathsf{ref}_C]\!] & = & \mathbb{W}(l : C, -) & & [\![A \to B]\!] & = & (\mathcal{T}_{\mathsf{fin}}[\![B]\!])^{[\![A]\!]} \end{array}$$

Fig. 5. Interpretation of λ_{ref} types in $\mathbf{W}$.

$\mathit{read}_C(l)$ is finitary since it only inspects a single location on the heap and full ground types have finite inhabitants. Formally we use the fact that sorts C with full ground interpretation can be enumerated (see Definition 4.8). Denote its enumeration as $?^{l:C} \xrightarrow{e} \Sigma_C$. By yoneda we have $\mathit{read}_C \in \mathcal{T}[\![\mathit{ctype}\, C]\!](l : C)$, and let its complete cover be e. The evaluation at every $\tau_i \in \Sigma_C$ is resultlet $\mathit{inj}_{\tau_i} \xrightarrow{1} \tau_i \langle \pi_{[e_i](l)} \eta_{e_i}, \eta_{e_i} \rangle$.

Generic Effect Write. Morphism $\mathit{write}_C : [\![\mathsf{ref}_C]\!] \times [\![\mathit{ctype}\, C]\!] \to \mathcal{T}\mathbb{1}$ is given as

$$\mathit{write}_C\langle l, d\rangle(w \xrightarrow{\sigma} w')(\eta : \mathbb{H}w') = q_{w' \xrightarrow{1} w'}\langle (), \eta[\sigma(l) \mapsto [\![\mathit{ctype}\, C]\!](\sigma)(d)]\rangle$$

$\mathit{write}_C\langle l, d\rangle \in \mathcal{T}\mathbb{1}w$ lifts through $\mathcal{T}_{\mathsf{fin}}$ monad as follows. Notice that l is a morphism $(l : C) \xrightarrow{l} w)$ and let $?^{l:C} \xrightarrow{e} \Sigma_C$ be an enumeration of C as before. We first rebase cover e (Definition 4.5) via $(l : C) \xrightarrow{l} w$, which induces a complete cover e^l over $\langle w, \Sigma_C^l \rangle$. Define the evaluation at every $\tau_i \in \Sigma_C^l$ as resultlet

$$\mathit{inj}_{\tau_i} \xrightarrow{1} \tau_i \langle (), \eta_{\varepsilon_i}[[e_i^l](l) \mapsto [\![\mathit{ctype}\, C]\!][e_i^l](d)]\rangle$$

Generic Effect Alloc. Recursive mutual allocation of locations is given by generic allocation *alloc*. For every world w_0 to be allocated, we have $\mathit{alloc}_{w_0} : \prod_{l:C \in w_0} [\![\mathit{ctype}\, C]\!](- \oplus w_0) \to \mathcal{T}\mathbb{W}(w_0, -)$. The extension functor $(- \oplus w_0)$ adds w_0-many locations, and so at a world w, we have w_0-many values $p \in \prod_{l:C \in w_0} [\![\mathit{ctype}\, C]\!](w \oplus w_0)$. Notice that p is equivalent to a morphism $w \xrightarrow{\varepsilon} w \oplus w_0$ in $\mathbb{W}\mathbb{I}$. Therefore, alloc_{w_0} is given as

$$\mathit{alloc}_{w_0}(p)(w \xrightarrow{\sigma} w')(\eta : \mathbb{H}w') = q_{\sigma^*\varepsilon}\langle \varepsilon^*\sigma \circ i_2, \mathbb{H}(\sigma^*\varepsilon)(\eta)\rangle$$

using the local independent coproduct $\begin{array}{ccc} w & \xrightarrow{\varepsilon} & w \oplus w_0 \\ \downarrow\sigma & & \downarrow\varepsilon^*\sigma \\ w' & \xrightarrow{\sigma^*\varepsilon} & w' \oplus_w (w \oplus w_0) \end{array}$ and notice that $\sigma^*\varepsilon$ can be equipped with the data of ε (see [15]).

We show that $\mathit{alloc}_{w_0}(p) \in \mathcal{T}\mathbb{W}(w_0, -)(w)$ is finitary by selecting the identity cover $?^w \overset{1}{\Rightarrow} ?^w$. For the resultlet, let $i_2^{w \oplus w_0} : w_0 \to w \oplus w_0$ be the template with $w \oplus w_0$ locations and $\mathsf{Img}(i_2)$ concrete ones. We have an allocation morphism $\rho : ?^w \to i_2^{w \oplus w_0}$ in $\mathbb{T}\mathbb{I}$. Define the resultlet as $\mathit{inj}_\rho\langle i_2, \eta_\varepsilon\rangle$.

5.3 Semantics of Terms

Typing judgments are interpreted as natural transformations in $\mathbf{W}$:

$$[\![\Gamma \vdash^{\mathsf{v}}_w M : A]\!]^{\mathsf{v}} : \mathbb{W}(w, -) \times [\![\Gamma]\!] \to [\![A]\!] \quad \text{and} \quad [\![\Gamma \vdash_w M : A]\!] : \mathbb{W}(w, -) \times [\![\Gamma]\!] \to \mathcal{T}_{\mathsf{fin}}[\![A]\!]$$

Contexts are interpreted component-wise $[\![\Gamma]\!] := \prod_{x:A \in \Gamma} [\![A]\!]$, and world w in judgment $\Gamma \vdash_w M : A$ is modelled using the representable functor $\mathbb{W}(w, -)$: it defines $[\![\Gamma \vdash_w M : A]\!]$ only at worlds w' that have an injection $w \rightarrowtail w'$, i.e., worlds bigger than w.

$$\begin{array}{rl} [\![\Gamma \vdash^{\mathsf{v}}_w V : A]\!]^{\mathsf{v}} : & \mathbb{W}(w,-) \times [\![\Gamma]\!] \to [\![A]\!] \\ [\![x : A, \Gamma \vdash^{\mathsf{v}}_w x : A]\!]^{\mathsf{v}}\langle\sigma,\gamma\rangle = \gamma(x) & [\![\Gamma \vdash^{\mathsf{v}}_w () : 1]\!]^{\mathsf{v}}\langle\sigma,\gamma\rangle = \star \\ [\![\Gamma \vdash^{\mathsf{v}}_w \lambda x.\, M : A \to B]\!]^{\mathsf{v}}\langle\sigma,\gamma\rangle = \mathrm{curry}[\![M]\!]\langle\sigma,\gamma\rangle & [\![\Gamma \vdash^{\mathsf{v}}_w (V,W) : A \times B]\!]^{\mathsf{v}}\langle\sigma,\gamma\rangle = \langle [\![V]\!]^{\mathsf{v}}, [\![W]\!]^{\mathsf{v}}\rangle\langle\sigma,\gamma\rangle \\ [\![\Gamma \vdash^{\mathsf{v}}_w l : \mathrm{ref}_C]\!]^{\mathsf{v}}\langle\sigma,\gamma\rangle = \sigma(l) & [\![\Gamma \vdash^{\mathsf{v}}_w \mathbf{inj}_i\, V : A + B]\!]^{\mathsf{v}}\langle\sigma,\gamma\rangle = inj_i([\![V]\!]^{\mathsf{v}}\langle\sigma,\gamma\rangle) \end{array}$$

Fig. 6. Interpretation of λ_{ref} values in $\mathbf{W}$

$$\begin{array}{rl} [\![\Gamma \vdash_w M : A]\!] : & \mathbb{W}(w,-) \times [\![\Gamma]\!] \to \mathcal{T}_{\mathsf{fin}}[\![A]\!] \\ [\![\Gamma \vdash_w V : A]\!] = & \eta_{[\![A]\!]} \circ [\![V]\!]^{\mathsf{v}} \\ [\![\Gamma \vdash_w M\, N : A]\!] = & \gg\!= \mathsf{eval} \circ \mathsf{dstr}_{[\![A \to B]\!],[\![A]\!]} \circ \langle [\![M]\!], [\![N]\!] \rangle \\ [\![\Gamma \vdash_w \mathbf{split}\, M\, \mathbf{as}\, (x_1,x_2).\, N : B]\!] = & \gg\!= [\![N]\!] \circ \mathsf{str} \circ \langle 1, [\![M]\!] \rangle \\ [\![\Gamma \vdash_w (M,N) : A \times B]\!] = & \mathsf{dstr} \circ \langle [\![M]\!], [\![N]\!] \rangle \\ [\![\Gamma \vdash_w \mathbf{inj}_i\, M : A + B]\!] = & \mathcal{T}_{\mathsf{fin}}(i_i) \circ [\![M]\!] \\ [\![\Gamma \vdash_w \mathbf{case}\, M\{\} : B]\!] = & \mathcal{T}_{\mathsf{fin}}(?_{[\![B]\!]}) \circ [\![M]\!] \\ [\![\Gamma \vdash_w \mathbf{case}\, M\, \{x_1.\, N_1; x_2.N_2\} : B]\!] = & \gg\!= [[\![N_1]\!], [\![N_2]\!]] \circ \mathcal{T}_{\mathsf{fin}}\mathsf{distr} \circ \mathsf{str} \circ \langle 1, [\![M]\!] \rangle \\ [\![\Gamma \vdash_w M == N : 2]\!] = & \mathcal{T}_{\mathsf{fin}} eq \circ \mathsf{dstr} \circ \langle [\![M]\!], [\![N]\!] \rangle \\ [\![\Gamma \vdash_w !\, M : \mathit{ctype}\, C]\!] = & \gg\!= \mathit{read} \circ [\![M]\!] \\ [\![\Gamma \vdash_w M := N : 1]\!] = & \gg\!= \mathit{write} \circ \mathsf{dstr} \circ \langle [\![M]\!], [\![N]\!] \rangle \\ [\![\Gamma \vdash_w \mathbf{letref}\, x_1 := V_1, \ldots, x_n := V_n\, \mathbf{in}\, M : B]\!]\langle\sigma,\gamma\rangle = & \mathsf{str}(\langle\sigma,\gamma\rangle, \mathit{alloc}\langle [\![V_i]\!]\langle\sigma \oplus w_0, \gamma'\rangle\rangle_{i=1}^n) \gg\!= [\![M]\!] \\ & \textit{where } \gamma' := [\![\Gamma]\!](i_1)(\gamma)[x_i \mapsto i_2(l_i)]_{i=1}^n \end{array}$$

Fig. 7. Interpretation of λ_{ref} expressions in $\mathbf{W}$

The interpretation of values (Figure 6 is mostly standard. It expects a location environment σ that associates location literals l with the current world, and a variable environment γ to interpret variables in context.

For the denotation of expressions (Figure 7), first denote the strength as $\mathrm{str}_{X,Y} : X \times \mathcal{T}_{\mathsf{fin}} Y \to \mathcal{T}_{\mathsf{fin}}(X \times Y)$, and the kleisli extension of a morphism $X \xrightarrow{N} \mathcal{T}_{\mathsf{fin}} Y$ as $\gg\!= N := \mu \circ \mathcal{T}_{\mathsf{fin}} N : \mathcal{T}_{\mathsf{fin}} X \to \mathcal{T}_{\mathsf{fin}} Y$. Further, define the left-to-right double strength dstr as (where $\mathrm{swap}_{X,Y} := \langle \pi_2, \pi_1 \rangle : X \times Y \to Y \times X$):

$$\mathsf{dstr}_{X,Y} \gg\!= (\mathsf{str} \circ \mathsf{swap}) \circ \mathsf{str} \circ \mathsf{swap} : \mathcal{T}_{\mathsf{fin}} X \times \mathcal{T}_{\mathsf{fin}} Y \to \mathcal{T}_{\mathsf{fin}}(X \times Y)$$

Lastly, let $?_X : \mathbb{0} \to X$ be the unique morphism from the initial object, and let $\mathrm{distr}_{X,Y,Z} : X \times (Y + Z) \to (X \times Y) + (X \times Z)$ be the distribution of products over sums given by the bi-ccc structure.

Semantically, a value $[\![\Gamma \vdash^{\mathsf{v}}_w W : A]\!]^{\mathsf{v}}$ is an expression $[\![\Gamma \vdash_w V : A]\!]$ by postcomposing with unit η. Constructs for applications, products and sums are interpreted in a standard way. For equality testing on references we use $eq : [\![\mathrm{ref}_C]\!] \times [\![\mathrm{ref}_C]\!] \to \mathbb{B}$ defined as $\langle \sigma_1, \sigma_2 \rangle \mapsto \sigma_1(l) == \sigma_2(l)$. For reading, writing and allocation we use the generic effects given at § 5.2. For allocation, we first lift environment $\gamma \in [\![\Gamma]\!](w')$ to $[\![\Gamma]\!](i_1)(\gamma) : [\![\Gamma]\!](w' \oplus w_0)$ and extend it to $\gamma' : [\![\Gamma, x_1 : \mathrm{ref}_C, \ldots, x_n : \mathrm{ref}_{C_n}]\!](w' \oplus w_0)$. We then apply *alloc* with values $[\![V_i]\!]\langle\sigma \oplus w_0, \gamma'\rangle : [\![\mathit{ctype}\, C]\!](w' \oplus w_0)$ where $\sigma \oplus w_0 : w \oplus w_0 \to w' \oplus w_0$.

5.4 Metatheory

Let $[\![-]\!]_{\mathcal{T}}$ denote the interpretation of λ_{ref} terms using the full monad $\mathcal{T}$, given identically to the one above. At full-ground types $D \in \mathsf{FGType}$, the denotations are identical $[\![D]\!]_{\mathcal{T}} = [\![D]\!]_{\mathcal{T}_{\text{fin}}}$, whereas at higher types they differ since $[\![A \to B]\!]_{\mathcal{T}} \neq [\![A \to B]\!]_{\mathcal{T}_{\text{fin}}}$.

Recall from § 4.3 that $\mathcal{T}_{\text{fin}}$ is related with $\mathcal{T}$ using a strong monad morphism ι. We can therefor establish a relationship between denotations of first-order terms given by the two monads [17].

Theorem 4. For every first-order term $x_1 : D_1, \dots, x_n : D_n \vdash_w M : D$ we have $\iota_{[\![D]\!]} \circ [\![M]\!]_{\mathcal{T}_{\text{fin}}} = [\![M]\!]_{\mathcal{T}}$.

Adequacy from the $[\![-]\!]_{\mathcal{T}}$ semantics extends to $[\![-]\!]_{\mathcal{T}_{\text{fin}}}$ using Theorem 4. Soundness of the semantics w.r.t. the operational semantics and compositionality is proven in the standard way by inducting over the derivations.

6 Related Work

Moggi [24] gave a unified categorical semantics for modelling various computational effects and specifically gave a possible worlds monad on a functor category for dynamic allocation of ground storage. Expanding on Moggi's work, Plotkin and Power [26] considered monads induced by suitable equations over the effectful operations, giving an equational account of computational effects. They further define a richer (ground) local state monad that supported garbage collection capabilities. Our work is based on the FGLS monad of Kammar et al. [15] that follows along this direction by giving a monad for *full ground* local state. Polzer and Goncharov [27] complement the semantics of [15] by developing a higher-order logic over the FGLS monad based on bi-hyperdoctorines.

Multiple relational models for local state exist in the literature that rely on auxiliary logical relations built on top of the semantics [5–7, 9–11]. These vary from utilising a continuation monad over nominal-cpos [7, 11], incorporating region-based effect typing in their reasoning [5, 6], and consider kripke logical relations over recursively defined set of worlds using metric spaces based semantics [10].

In any case, the relative simplicity of the underlying denotational semantics means that they fail to validate desired equivalences involving unobservable allocations. Instead it allows them to model complex features such as general references, recursive types and parametric polymorphism. They employ logical relations to refine their model allowing them to reason about properties such as observational equivalence. In contrast, the possible worlds semantics of [15, 26] builds garbage collection into the model, complicating the model but avoid requiring auxiliary logical relations.

Syntactic step-indexed logical relations built over the operational semantics have also been successful in modelling multiple non-trivial programming language constructs in conjunction such as recursive, abstract, polymorphic types, general references and control effects [2, 3, 9, 12]. By marrying the reasoning with separation logic [14, 34] they are able to prove deep properties of higher-order concurrent programs that make use of general references. We instead focus on denotational accounts of local state that abstract away from lower-level operational considerations.

An alternative class of models is based on game semantics, which differs from the aforementioned models as the semantics of heaps is implicit [1, 19, 25]. In this line of work, execution of a program is modelled as a dialogue between the player representing the program and the opponent being the environment. Game semantics is suitable for modelling full ground and general references and produces models that are fully abstract at higher-order types. Nevertheless, these models are fundamentally different from the possible worlds semantics we are concerned with.

7 Conclusion and Future Work

We extend the metatheory of the FGLS monad introduced in [15] by resolving an outstanding question on whether it is finitary. We show a negative result and therefore introduce a finitary

FGLS submonad. This submonad paves the way towards finitary equational axiomatisations of FGLS that are more amenable to reasoning.

There are several directions for future work. A useful property for a semantics is full abstraction at first order types [1, 16, 22, 33]. It states that for terms $\Gamma \vdash M, N : A$, observational equivalence implies denotational equality: $M \cong_{\mathrm{obs}} N \implies [\![M]\!] = [\![N]\!]$, where Γ, A are first order, i.e., do not have any function types. Together with adequacy, full abstraction at first order identifies observational and denotational equality of first order terms. Such a property is used to show that the model is complete for compiler optimisations on imperative programs involving full ground references. This is typically the best-possible result, as models given by sets with structure and structure preserving functions are not typically fully abstract at higher order. We conjecture that the FGLS monad is fully abstract at first order, and that we can use techniques inspired by game semantics to prove that this is the case. This result would then transfer to the finitary submonad, and we can exploit its finitary nature to develop decision procedures to decide whether two terms are observationally equivalent.

We also aim to generalise the equations for ground local state [26, 31, 32] to the full-ground setting. This is a non-trivial task for two reasons: (i) one must identify an appropriate framework in which heap values can be treated as first-class entities, and (ii) one must account for the symmetries present within an allocated block of data. The goal is to obtain an equational axiomatisation giving rise to the finitary FGLS monad developed here.

Furthermore, we aim to increase the expressivity of the FGLS monad by reformulating it over $\mathbb{W}$-indexed functors into cpos. This would allow us to support general recursion and would be a further step towards modelling higher-order store. As an additional benefit, such a domain-theoretic presentation of the FGLS monad may offer an alternative characterisation of the finitary submonad in terms of abstract domain constructions.

Finally, it would be worthwhile to investigate which programming-language constructs can express complete heap traversals without introducing a primitive operation that returns the size of the heap, such as *heapSize* from § 2.3. We expect, however, that such an operation would have no substantial impact on the metatheory: the FGLS monad would still fail to be fully abstract at higher-order types.

Acknowledgments

Ohad Kammar was funded by a Royal Society University Research Fellowship, Sam Lindley was supported by a UKRI Future Leaders Fellowship “Effect Handler Oriented Programming” (MR/T043830/1 and MR/Z000351/1), and Cristina Matache was partly supported by a Leverhulme Early Career Fellowship. We thank the Paul B. Levy for useful discussions and suggestions.

References

[1] Samson Abramsky, Kohei Honda, and Guy McCusker. 1998. A Fully Abstract Game Semantics for General References. In *IEEE Symposium on Logic in Computer Science*. IEEE Computer Society, Indiana, USA, 334–344. doi:10.1109/LICS.1998.705669

[2] Amal Ahmed, Derek Dreyer, and Andreas Rossberg. 2009. State-dependent representation independence. In *POPL*. ACM, Savannah, GA, USA, 340–353. doi:10.1145/1480881.1480925

[3] Amal J. Ahmed, Matthew Fluet, and Greg Morrisett. 2005. A step-indexed model of substructural state. In *ICFP*. ACM, Tallinn, Estonia, 78–91. doi:10.1145/1086365.1086376

[4] Andrej Bauer. 2018. What is algebraic about algebraic effects and handlers? *CoRR* abs/1807.05923 (2018). arXiv:1807.05923 http://arxiv.org/abs/1807.05923

[5] Nick Benton, Martin Hofmann, and Vivek Nigam. 2014. Abstract effects and proof-relevant logical relations. In *POPL*. ACM, San Diego, CA, USA, 619–632. doi:10.1145/2535838.2535869

[6] Nick Benton, Andrew Kennedy, Lennart Beringer, and Martin Hofmann. 2007. Relational semantics for effect-based program transformations with dynamic allocation. In *PPDP*. ACM, Wroclaw, Poland, 87–96. doi:10.1145/1273920.

1273932
[7] Nick Benton and Benjamin Leperchey. 2005. Relational Reasoning in a Nominal Semantics for Storage. In *TLCA (Lecture Notes in Computer Science, Vol. 3461)*, Pawel Urzyczyn (Ed.). Springer, Nara, Japan, 86–101. doi:10.1007/11417170_8
[8] Clemens Berger, Paul-André Melliès, and Mark Weber. 2012. Monads with arities and their associated theories. *Journal of Pure and Applied Algebra* 216, 8 (2012), 2029–2048. Special Issue devoted to the International Conference in Category Theory 'CT2010'. doi:10.1016/j.jpaa.2012.02.039
[9] Lars Birkedal, Bernhard Reus, Jan Schwinghammer, Kristian Støvring, Jacob Thamsborg, and Hongseok Yang. 2011. Step-indexed Kripke models over recursive worlds. *ACM sigplan notices* 46, 1 (2011), 119–132.
[10] Lars Birkedal, Kristian Støvring, and Jacob Thamsborg. 2010. Realisability semantics of parametric polymorphism, general references and recursive types. *Math. Struct. Comput. Sci.* 20, 4 (2010), 655–703. doi:10.1017/S0960129510000162
[11] Nina Bohr and Lars Birkedal. 2006. Relational Reasoning for Recursive Types and References. In *APLAS (Lecture Notes in Computer Science, Vol. 4279)*, Naoki Kobayashi (Ed.). Springer, Sydney, Australia, 79–96. doi:10.1007/11924661_5
[12] Derek Dreyer, Georg Neis, and Lars Birkedal. 2012. The impact of higher-order state and control effects on local relational reasoning. *J. Funct. Program.* 22, 4-5 (2012), 477–528. doi:10.1017/S095679681200024X
[13] Martin Hyland and John Power. 2007. The Category Theoretic Understanding of Universal Algebra: Lawvere Theories and Monads. 172 (2007), 437–458. doi:10.1016/J.ENTCS.2007.02.019
[14] Ralf Jung, Robbert Krebbers, Jacques-Henri Jourdan, Ales Bizjak, Lars Birkedal, and Derek Dreyer. 2018. Iris from the ground up: A modular foundation for higher-order concurrent separation logic. *J. Funct. Program.* 28 (2018), e20. doi:10.1017/S0956796818000151
[15] Ohad Kammar, Paul Blain Levy, Sean K. Moss, and Sam Staton. 2017. A monad for full ground reference cells. In *LICS*. IEEE Computer Society, Reykjavik, Iceland, 1–12. doi:10.1109/LICS.2017.8005109
[16] Ohad Kammar, Jack Liell-Cock, Sam Lindley, Cristina Matache, and Sam Staton. 2026. An Equational Axiomatization of Dynamic Threads via Algebraic Effects: Presheaves on Finite Relations, Labelled Posets, and Parameterized Algebraic Theories. *Proc. ACM Program. Lang.* 10, POPL (2026), 1847–1875. doi:10.1145/3776706
[17] Shin-ya Katsumata. 2013. Relating computational effects by $\top\top$-lifting. *Information and Computation* 222 (2013), 228–246.
[18] Gregory M Kelly. 1982. Structures defined by finite limits in the enriched context, I. *Cahiers de Topologie et Géométrie Différentielle* 23, 1 (1982), 3–42.
[19] J. Laird. 2008. A game semantics of names and pointers. *Annals of Pure and Applied Logic* 151, 2 (2008), 151–169. doi:10.1016/j.apal.2007.10.006
[20] P. J. Landin. 1964. The Mechanical Evaluation of Expressions. *Comput. J.* 6, 4 (1964), 308–320. doi:10.1093/COMJNL/6.4.308
[21] Fred EJ Linton. 1966. Some aspects of equational categories. In *Proceedings of the Conference on Categorical Algebra*. Springer, Berlin, Heidelberg, 84–94.
[22] Robin Milner. 1977. Fully Abstract Models of Typed *lambda*-Calculi. *Theor. Comput. Sci.* 4, 1 (1977), 1–22. doi:10.1016/0304-3975(77)90053-6
[23] Eugenio Moggi. 1989. Computational Lambda-Calculus and Monads. In *LICS*. IEEE Computer Society, Pacific Grove, California, USA, 14–23. doi:10.1109/LICS.1989.39155
[24] Eugenio Moggi. 1990. *An abstract view of programming languages*. University of Edinburgh. Department of Computer Science, Edinburgh,UK.
[25] Andrzej S. Murawski and Nikos Tzevelekos. 2012. Algorithmic Games for Full Ground References. In *ICALP (Lecture Notes in Computer Science, Vol. 7392)*. Springer, Warwick, UK, 312–324. doi:10.1007/978-3-642-31585-5_30
[26] Gordon D. Plotkin and John Power. 2002. Notions of Computation Determine Monads. In *FOSSACS (Lecture Notes in Computer Science, Vol. 2303)*. Springer, Grenoble, France, 342–356. doi:10.1007/3-540-45931-6_24
[27] Miriam Polzer and Sergey Goncharov. 2020. Local local reasoning: A bi-hyperdoctrine for full ground store. In *FoSSaCS*. Springer, Springer International Publishing, Cham, 542–561.
[28] John Power. 1999. Enriched lawvere theories. *Theory and Applications of Categories* 6, 7 (1999), 83–93.
[29] Edmund Robinson. 2002. Variations on algebra: Monadicity and generalisations of equational therories. *Formal Aspects of Computing* 13, 3 (2002), 308–326.
[30] Alex Simpson. 2018. Category-theoretic structure for independence and conditional independence. *Electronic Notes in Theoretical Computer Science* 336 (2018), 281–297.
[31] Sam Staton. 2010. Completeness for Algebraic Theories of Local State. In *FOSSACS (Lecture Notes in Computer Science, Vol. 6014)*, C.-H. Luke Ong (Ed.). Springer, Paphos, Cyprus, 48–63. doi:10.1007/978-3-642-12032-9_5
[32] Sam Staton. 2013. Instances of Computational Effects: An Algebraic Perspective. In *LICS*. IEEE Computer Society, New Orleans, LA, USA, 519. doi:10.1109/LICS.2013.58
[33] Thomas Streicher. 2006. *Domain-theoretic foundations of functional programming*. World Scientific, Toh Tuck Link, Singapore.

[34] Amin Timany, Robbert Krebbers, Derek Dreyer, and Lars Birkedal. 2024. A Logical Approach to Type Soundness. *J. ACM* 71, 6 (2024), 40:1–40:75. doi:10.1145/3676954

A Appendix

A.1 Full Specification of $\lambda_{\mathbf{ref}}$

$$
\begin{array}{rll}
\text{Type} \ni A, B ::= & \boxed{\mathsf{ref}_C} \mid 0 \mid A + B \mid 1 \mid A \times B \mid A \to B & \\
\text{Val} \ni V, W ::= & x \mid \lambda x.\, M \mid & (\lambda\text{-calculus}) \\
& \boxed{l} \mid & (\text{Location Literals}) \\
& () \mid (V, W) \mid & (\text{Products}) \\
& \mathbf{inj}_{i \in \{1,2\}}\ V & (\text{Sums}) \\
\text{Expr} \ni M, N ::= & V \mid M\, N \mid & (\text{Values and Application}) \\
& (M, N) \mid \mathbf{split}\, M\, \mathbf{as}\, (x_1, x_2).\, N \mid & (\text{Products}) \\
& \mathbf{inj}_{i \in \{1,2\}}\ M \mid \mathbf{case}\, M \{\} \mid \mathbf{case}\, M\, \{x_1.\, N_1; x_2.N_2\} \mid & (\text{Sums}) \\
& \boxed{!\, M} \mid \boxed{M := N} \mid \boxed{\mathbf{letref}\ x_1 := V_1, \ldots, x_n := V_n\ \mathbf{in}\ M} & (\text{References})
\end{array}
$$

Fig. 8. Syntax of λ_{ref}

$$
\textsc{Var}\ \frac{x : A \in \Gamma}{\Gamma \vdash^{\mathsf{v}}_w x : A} \qquad
\textsc{Lam}\ \frac{\Gamma, x : A \vdash_w M : B}{\Gamma \vdash^{\mathsf{v}}_w \lambda x.\, M : A \to B} \qquad
\boxed{\textsc{Loc}\ \frac{l : C \in w}{\Gamma \vdash^{\mathsf{v}}_w l : \mathsf{ref}_C}} \qquad
\textsc{Unit}\ \Gamma \vdash^{\mathsf{v}}_w () : 1 \qquad
\textsc{VPair}\ \frac{\Gamma \vdash^{\mathsf{v}}_w V : A \quad \Gamma \vdash^{\mathsf{v}}_w W : B}{\Gamma \vdash^{\mathsf{v}}_w (V, W) : A \times B}
$$

$$
\textsc{VInj}\ \frac{\Gamma \vdash^{\mathsf{v}}_w V : A_i}{\Gamma \vdash^{\mathsf{v}}_w \mathbf{inj}_i\, V : A_1 + A_2} \qquad
\textsc{Val}\ \frac{\Gamma \vdash^{\mathsf{v}}_w x : A}{\Gamma \vdash_w x : A} \qquad
\textsc{App}\ \frac{\Gamma \vdash^{\mathsf{v}}_w M : A \to B \quad \Gamma \vdash^{\mathsf{v}}_w N : A}{\Gamma \vdash_w M\, N : B} \qquad
\textsc{EPair}\ \frac{\Gamma \vdash_w M : A \quad \Gamma \vdash_w N : B}{\Gamma \vdash_w (M, N) : A \times B}
$$

$$
\textsc{Split}\ \frac{\Gamma \vdash_w M : A_1 \times A_2 \quad \Gamma, x_1 : A_1, x_2 : A_2 \vdash_w N : B}{\Gamma \vdash_w \mathbf{split}\, M\, \mathbf{as}\, (x, y).\, N : B} \qquad
\textsc{EInj}\ \frac{\Gamma \vdash_w M : A_i}{\Gamma \vdash_w \mathbf{inj}_i\, M : A_1 + A_2} \qquad
\textsc{Case}\ \frac{\Gamma \vdash_w M : 0}{\Gamma \vdash_w \mathbf{case}\, M\{\} : A}
$$

$$
\textsc{Case}\ \frac{\Gamma \vdash_w M : A_1 + A_2 \quad (\Gamma, x_i : A_i \vdash_w N_i : B)_{i=1}^2}{\Gamma \vdash_w \mathbf{case}\, M\, \{x_1.\, N_1; x_2.N_2\} : B} \qquad
\boxed{\textsc{Read}\ \frac{\Gamma \vdash_w M : \mathsf{ref}_C}{\Gamma \vdash_w !\, M : \mathit{ctype}\, C}} \qquad
\boxed{\textsc{Write}\ \frac{\Gamma \vdash_w M : \mathsf{ref}_C \quad \Gamma \vdash_w N : \mathit{ctype}\, C}{\Gamma \vdash_w M := N : 1}}
$$

$$
\boxed{\textsc{Alloc}\ \frac{(\Gamma, x_1 : \mathsf{ref}_{C_1}, \ldots, x_n : \mathsf{ref}_{C_n} \vdash_w V_i : \mathit{ctype}\, C_i)_{i=1}^n \quad \Gamma, x_1 : \mathsf{ref}_{C_1}, \ldots, x_n : \mathsf{ref}_{C_n} \vdash_w M : A}{\Gamma \vdash_w \mathbf{letref}\ x_1 := V_1, \ldots, x_n := V_n\ \mathbf{in}\ M : A}}
$$

Fig. 9. Typing rules of λ_{ref}

$$\textsc{Red-Val}\quad \langle V, \eta\rangle \Downarrow \langle V, \eta\rangle$$

$$\textsc{Red-App}\quad \frac{\langle M, \eta\rangle \Downarrow \langle \lambda x.\, M', \eta'\rangle \qquad \langle N, \eta'\rangle \Downarrow \langle V, \eta''\rangle \qquad \langle M'[V/x], \eta''\rangle \Downarrow \langle W, \eta'''\rangle}{\langle M\, N, \eta\rangle \Downarrow \langle W, \eta'''\rangle}$$

$$\textsc{Red-EPair}\quad \frac{\langle M, \eta\rangle \Downarrow \langle V, \eta'\rangle \qquad \langle N, \eta'\rangle \Downarrow \langle W, \eta''\rangle}{\langle (M, N), \eta\rangle \Downarrow \langle (V, W), \eta''\rangle}$$

$$\textsc{Red-Split}\quad \frac{\langle M, \eta\rangle \Downarrow \langle (V_1, V_2), \eta'\rangle \qquad \langle N[x_1/V_1, x_2/V_2], \eta'\rangle \Downarrow \langle W, \eta''\rangle}{\langle \mathbf{split}\, M\, \mathbf{as}\, (x_1, x_2).\, N, \eta\rangle \Downarrow \langle W, \eta''\rangle}$$

$$\textsc{Red-EInj}\quad \frac{\langle M, \eta\rangle \Downarrow \langle V, \eta'\rangle}{\langle \mathbf{inj}_i\, M, \eta\rangle \Downarrow \langle \mathbf{inj}_i\, V, \eta'\rangle}$$

$$\textsc{Red-Case}\quad \frac{\langle M, \eta\rangle \Downarrow \langle \mathbf{inj}_i\, V, \eta'\rangle \qquad \langle M_i[x_i/V], \eta'\rangle \Downarrow \langle W, \eta''\rangle}{\langle \mathbf{case}\, M\, \{x_1.\, N_1; x_2.N_2\}, \eta\rangle \Downarrow \langle W, \eta''\rangle}$$

$$\textsc{Red-Read}\quad \frac{\langle M, \eta\rangle \Downarrow \langle l, \eta'\rangle \qquad \eta'(l) = V}{\langle !\, M, \eta\rangle \Downarrow \langle V, \eta'\rangle}$$

$$\textsc{Red-Write}\quad \frac{\langle M, \eta\rangle \Downarrow \langle l, \eta'\rangle \qquad \langle N, \eta'\rangle \Downarrow \langle W, \eta''\rangle}{\langle M := N, \eta\rangle \Downarrow \langle (), \eta''[l \mapsto W]\rangle}$$

$$\textsc{Red-Eq}\quad \frac{\langle M, h\rangle \Downarrow \langle l_1, h'\rangle \qquad \langle N, h'\rangle \Downarrow \langle l_2, h''\rangle}{\langle M == N, h\rangle \Downarrow \langle \#(l_1 == l_2), h''\rangle}$$

$$\textsc{Red-Alloc}\quad \frac{l_1, \ldots, l_n \notin \mathrm{Dom}(\eta) \qquad \theta = [x_i \mapsto l_i]_{i=1}^n \qquad \eta' = \eta[l_i \mapsto V_i[\theta]]_{i=1}^n \qquad \langle M[\theta], \eta'\rangle \Downarrow \langle W, \eta''\rangle}{\langle \mathbf{letref}\, x_1 := V_1, \ldots, x_n := V_n\, \mathbf{in}\, M, \eta\rangle \Downarrow \langle W, \eta''\rangle}$$

Fig. 10. Reduction rules of λ_{ref} in big-step

A.2 Non-Finitariness of FGLS Monad

Lemma A.1. Monad $\mathcal{T}$ does not preserve filtered colimits.

Proof. Recall the local state function *heapSize* from § 2.3, and for simplicity, assume we have only one sort nat. It is straightforward to extend *heapSize* to any interpretation function defined over the full ground types of § 2.1.

heapSize takes a location l : nat, traverses the heap starting from l and returns the number of distinct locations it has found. Let $\Delta_{\mathbb{N}} : \mathbb{W} \to \mathrm{Set}$ be the constant functor $\Delta_{\mathbb{N}}\, w = \mathbb{N}$ in

$$\begin{aligned}
&\mathit{heapSize} : \int_{l:\mathrm{nat} \to w \in l:\mathrm{nat}/U} \mathbb{H}w \to \mathbb{P}(\Delta_{\mathbb{N}} \times \mathbb{H})(w)\\
&\mathit{heapSize}((l : \mathrm{nat}) \xrightarrow{\rho} w)(h : \mathbb{H}w) = q_{w \le w}\langle n, h\rangle\\
&\textit{where}\\
&\quad go = \lambda\langle l \in w, S \subseteq w\rangle.\, \mathbf{case}\, \pi_l h\, \mathbf{of}\\
&\qquad\qquad \mathbf{inl}\, ().\, 1\\
&\qquad\qquad \mathbf{inr}\, b.\, \begin{cases} 1 & \mathit{if}\ b \notin S\\ 1 + go(b, S \setminus \{b\}) & \mathit{otherwise}\end{cases}\\
&\quad n = go\langle \rho(l), w\rangle
\end{aligned}$$

Notice that *go* is well-founded since S strictly reduces in size at every recursive call. It also satisfies the coherence condition of the End as it only traverses the reachable part of the heap w.r.t. public location l.

To show that *heapSize* is a counterexample to $\mathcal{T}$ being finitary we need a filtered colimit that is not preserved by $\mathcal{T}$. Consider the (filtered) posetal category ω of natural numbers with the usual ordering, and let $\Delta_{_} : \mathit{colimit}\,(\omega \to [\mathbb{W}, \mathrm{Set}])$ be the functor that maps each n to its down-set, $\Delta_n = [n]$. $\mathcal{T}$ does not preserve the (filtered) $\mathit{colimit}\,\Delta_{_} : \mathbb{W} \to \mathrm{Set}$ because the mapping given by the appropriate inclusion

$$\biguplus_{n \in \mathbb{N}} \mathcal{T}(\Delta_{[n]})(w) \to \mathcal{T}(\Delta_{\mathbb{N}})(w)$$

is not surjective: there is no n that defines *heapSize* □